```
# ============================================================
# PMBSF_v17_BS_smooth_source_connected_cumulants.py
#
# Purpose:
#   Numerical diagnostic for BS = Balaban smooth-source expansion.
#
#   Estimates connected square-free source coefficients / cumulants
#   for smoothed high-plaquette observables:
#
#       X_{p,eta} = sigmoid((phi(U_p)-t)/eta)
#
#   where phi(U_p) = 1 - (1/2) ReTr(U_p) = 1 - q0(U_p).
#
#   This does NOT prove BS. It tests the finite-volume numerical shadow:
#
#       |kappa(B)| / q_eta^{|B|}
#
#   versus support geometry / tree distance.
#
# Recommended runtime:
#   A100 GPU.
#
# Notes:
#   - Uses SU(2) quaternion Metropolis, not exact heatbath.
#   - Saves thermalized fields to avoid repeating expensive thermalization.
#   - Works in fixed-threshold smooth-source mode, not top-p mode.
# ============================================================

import os
import math
import json
import time
import zipfile
import random
from dataclasses import dataclass
from typing import Dict, List, Tuple

import numpy as np
import pandas as pd
import torch


# ============================================================
# User configuration
# ============================================================

RUN_TAG = "PMBSF_v17_BS_smooth_source_connected_cumulants"

# Use "pilot" first. Use "a100" for serious run.
RUN_MODE = "pilot"   # "pilot" or "a100"

if RUN_MODE == "pilot":
    L_LIST = [12]
    BETA_LIST = [3.5]
    START_MODE = "hot"
    THERMAL_SWEEPS = 100
    N_CALIB_CFG = 8
    N_MEAS_CFG = 30
    BETWEEN_SWEEPS = 10
    ETA_LIST = [0.05, 0.10]
    Q_TARGET_LIST = [0.003, 0.01]
    R_DISTANCES = [1, 2, 3, 4]
    CALIB_MAX_PHI_PER_CFG = 150_000
else:
    L_LIST = [12, 16, 24]
    BETA_LIST = [3.5, 4.0]
    START_MODE = "hot"
    THERMAL_SWEEPS = 400
    N_CALIB_CFG = 20
    N_MEAS_CFG = 200
    BETWEEN_SWEEPS = 20
    ETA_LIST = [0.025, 0.05, 0.10]
    Q_TARGET_LIST = [0.001, 0.003, 0.01]
    R_DISTANCES = [1, 2, 3, 4, 6, 8]
    CALIB_MAX_PHI_PER_CFG = 300_000
```

```
BASE_PROPOSAL_SIGMA = None  # if None: 0.80 / sqrt(beta)

ADAPT_PROPOSAL = True
ADAPT_EVERY = 10
ADAPT_TARGET_ACC = 0.55
ADAPT_RATE = 0.08

SEED = 23061701

BASE_OUTDIR = "/content/PMBSF_v17_BS_output"
FIELD_CACHE_DIR = "/content/PMBSF_v17_thermalized_fields"
SAVE_FIELD_CACHE = True
LOAD_FIELD_CACHE = True

DTYPE = torch.float32
SIGMOID_CLIP = 60.0
EPS = 1e-30


# ============================================================
# Device setup
# ============================================================

def get_device():
    if torch.cuda.is_available():
        dev = torch.device("cuda")
        print(f"[device] CUDA: {torch.cuda.get_device_name(0)}")
        return dev
    print("[device] CPU fallback")
    return torch.device("cpu")


DEVICE = get_device()
torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)

os.makedirs(BASE_OUTDIR, exist_ok=True)
os.makedirs(FIELD_CACHE_DIR, exist_ok=True)

RUN_ID = f"{RUN_TAG}_{time.strftime('%Y%m%d_%H%M%S')}"
OUTDIR = os.path.join(BASE_OUTDIR, RUN_ID)
os.makedirs(OUTDIR, exist_ok=True)

print(f"[run] {RUN_ID}")
print(f"[outdir] {OUTDIR}")


# ============================================================
# SU(2) quaternion algebra
# Quaternion q = (a0, a1, a2, a3)
# ============================================================

def qmul(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    a0, a1, a2, a3 = a.unbind(dim=-1)
    b0, b1, b2, b3 = b.unbind(dim=-1)
    return torch.stack(
        (
            a0*b0 - a1*b1 - a2*b2 - a3*b3,
            a0*b1 + a1*b0 + a2*b3 - a3*b2,
            a0*b2 - a1*b3 + a2*b0 + a3*b1,
            a0*b3 + a1*b2 - a2*b1 + a3*b0,
        ),
        dim=-1,
    )


def qconj(a: torch.Tensor) -> torch.Tensor:
    out = a.clone()
    out[..., 1:] *= -1
    return out


def qnorm(a: torch.Tensor) -> torch.Tensor:
    return torch.sqrt(torch.sum(a * a, dim=-1, keepdim=True).clamp_min(EPS))
```

```
def qnormalize(a: torch.Tensor) -> torch.Tensor:
    return a / qnorm(a)


def random_su2(shape, device=DEVICE, dtype=DTYPE) -> torch.Tensor:
    q = torch.randn((*shape, 4), device=device, dtype=dtype)
    return qnormalize(q)


def cold_su2_links(L: int) -> torch.Tensor:
    U = torch.zeros((4, L, L, L, L, 4), device=DEVICE, dtype=DTYPE)
    U[..., 0] = 1.0
    return U


def hot_su2_links(L: int) -> torch.Tensor:
    return random_su2((4, L, L, L, L), device=DEVICE, dtype=DTYPE)


def random_near_identity(shape, sigma: float) -> torch.Tensor:
    v = sigma * torch.randn((*shape, 3), device=DEVICE, dtype=DTYPE)
    q = torch.cat(
        [torch.ones((*shape, 1), device=DEVICE, dtype=DTYPE), v],
        dim=-1,
    )
    return qnormalize(q)


def shift_field(F: torch.Tensor, shift: Tuple[int, int, int, int]) -> torch.Tensor:
    """
    Return F(x + shift), aligned at x.
    """
    return torch.roll(F, shifts=tuple(-s for s in shift), dims=(0, 1, 2, 3))


def unit_shift(mu: int) -> Tuple[int, int, int, int]:
    s = [0, 0, 0, 0]
    s[mu] = 1
    return tuple(s)


def add_shift(a, b):
    return tuple(int(a[i] + b[i]) for i in range(4))


# ============================================================
# Gauge action / Metropolis update
# ============================================================

PLAQUETTE_ORIS = [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]


def staple_for_link(U: torch.Tensor, mu: int) -> torch.Tensor:
    """
    Staple V_mu(x) for local Wilson action contribution:
        - beta * scalar_part(U_mu(x) * V_mu(x))
    """
    L = U.shape[1]
    V = torch.zeros((L, L, L, L, 4), device=U.device, dtype=U.dtype)

    emu = unit_shift(mu)

    for nu in range(4):
        if nu == mu:
            continue

        enu = unit_shift(nu)

        U_nu_xpmu = shift_field(U[nu], emu)
        U_mu_xpnu = shift_field(U[mu], enu)
        U_nu_x = U[nu]

        # Positive staple:
        # U_nu(x+mu) U_mu^\dagger(x+nu) U_nu^\dagger(x)
        V_pos = qmul(qmul(U_nu_xpmu, qconj(U_mu_xpnu)), qconj(U_nu_x))

        # Negative staple:
```

```
        # U_nu^\dagger(x-nu+mu) U_mu^\dagger(x-nu) U_nu(x-nu)
        minus_enu = tuple(-x for x in enu)
        shift_munu = add_shift(emu, minus_enu)

        U_nu_xmnu_pmu = shift_field(U[nu], shift_munu)
        U_mu_xmnu = shift_field(U[mu], minus_enu)
        U_nu_xmnu = shift_field(U[nu], minus_enu)

        V_neg = qmul(qmul(qconj(U_nu_xmnu_pmu), qconj(U_mu_xmnu)), U_nu_xmnu)

        V = V + V_pos + V_neg

    return V


def make_parity_mask(L: int, parity: int) -> torch.Tensor:
    coords = torch.meshgrid(
        torch.arange(L, device=DEVICE),
        torch.arange(L, device=DEVICE),
        torch.arange(L, device=DEVICE),
        torch.arange(L, device=DEVICE),
        indexing="ij",
    )
    s = coords[0] + coords[1] + coords[2] + coords[3]
    return (s % 2) == parity


def metropolis_sweep(U: torch.Tensor, beta: float, sigma: float, parity_masks) -> float:
    accs = []

    for mu in range(4):
        for parity in (0, 1):
            mask = parity_masks[parity]

            V = staple_for_link(U, mu)
            old_score = qmul(U[mu], V)[..., 0]

            R = random_near_identity(U[mu].shape[:-1], sigma)
            U_prop = qnormalize(qmul(R, U[mu]))
            new_score = qmul(U_prop, V)[..., 0]

            log_accept_ratio = beta * (new_score - old_score)
            logu = torch.log(torch.rand_like(log_accept_ratio).clamp_min(EPS))
            accept = (logu < log_accept_ratio) & mask

            U[mu] = torch.where(accept[..., None], U_prop, U[mu])

            if mask.any():
                accs.append(accept[mask].float().mean().item())

    return float(np.mean(accs))


def plaquette_phi(U: torch.Tensor) -> torch.Tensor:
    """
    Return phi for all six plaquette orientations.
    Shape: [6, L, L, L, L]
    phi = 1 - scalar_part(U_p)
    """
    phis = []

    for (mu, nu) in PLAQUETTE_ORIS:
        emu = unit_shift(mu)
        enu = unit_shift(nu)

        U_mu_x = U[mu]
        U_nu_xpmu = shift_field(U[nu], emu)
        U_mu_xpnu = shift_field(U[mu], enu)
        U_nu_x = U[nu]

        P = qmul(
            qmul(qmul(U_mu_x, U_nu_xpmu), qconj(U_mu_xpnu)),
            qconj(U_nu_x),
        )
        q0 = P[..., 0].clamp(-1.0, 1.0)
        phi = (1.0 - q0).clamp(0.0, 2.0)
        phis.append(phi)
```

```
    return torch.stack(phis, dim=0)


# ============================================================
# Smooth threshold calibration
# ============================================================

def sigmoid_np(z):
    z = np.clip(z, -SIGMOID_CLIP, SIGMOID_CLIP)
    return 1.0 / (1.0 + np.exp(-z))


def calibrate_threshold_from_samples(phi_samples: np.ndarray, eta: float, q_target: float) -> float:
    """
    Find t so mean sigmoid((phi-t)/eta) = q_target.
    """
    samples = phi_samples.astype(np.float64)
    lo = float(samples.min() - 20.0 * eta)
    hi = float(samples.max() + 20.0 * eta)

    for _ in range(90):
        mid = 0.5 * (lo + hi)
        m = sigmoid_np((samples - mid) / eta).mean()
        if m > q_target:
            lo = mid
        else:
            hi = mid

    return float(0.5 * (lo + hi))


def smooth_X_from_phi(phi: torch.Tensor, threshold: float, eta: float) -> torch.Tensor:
    z = ((phi - threshold) / eta).clamp(-SIGMOID_CLIP, SIGMOID_CLIP)
    return torch.sigmoid(z)


# ============================================================
# Pattern / cumulant machinery
# ============================================================

@dataclass(frozen=True)
class PlaqRef:
    ori: int
    shift: Tuple[int, int, int, int]


@dataclass
class Pattern:
    name: str
    refs: List[PlaqRef]
    kind: str


def pair_dist(a: PlaqRef, b: PlaqRef) -> int:
    d = sum(abs(a.shift[i] - b.shift[i]) for i in range(4))
    if a.ori != b.ori:
        d += 1
    return int(d)


def mst_tau(refs: List[PlaqRef]) -> int:
    n = len(refs)
    if n <= 1:
        return 0

    used = [False] * n
    used[0] = True
    total = 0

    for _ in range(n - 1):
        best = None
        for i in range(n):
            if not used[i]:
                continue
            for j in range(n):
                if used[j]:
                    continue
```

```
                d = pair_dist(refs[i], refs[j])
                if best is None or d < best[0]:
                    best = (d, j)
        total += best[0]
        used[best[1]] = True

    return int(total)


def build_patterns(L: int) -> List[Pattern]:
    e0 = (1, 0, 0, 0)
    e1 = (0, 1, 0, 0)

    patterns = []

    for r in R_DISTANCES:
        if r < L // 2:
            patterns.append(Pattern(
                name=f"pair_same_ori_axis_r{r}",
                kind="pair_same_ori",
                refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(0, (r, 0, 0, 0))]
            ))
            patterns.append(Pattern(
                name=f"pair_same_ori_diag_r{r}",
                kind="pair_same_ori_diag",
                refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(0, (r, r, 0, 0))]
            ))

    patterns.extend([
        Pattern(
            name="pair_incident_same_site_01_02",
            kind="pair_incident",
            refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, (0, 0, 0, 0))]
        ),
        Pattern(
            name="pair_incident_same_site_01_03",
            kind="pair_incident",
            refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(2, (0, 0, 0, 0))]
        ),
        Pattern(
            name="pair_near_shifted_mixed",
            kind="pair_mixed_near",
            refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, (1, 0, 0, 0))]
        ),
    ])

    far_r = max(2, L // 3)

    patterns.append(Pattern(
        name=f"pair_far_control_r{far_r}",
        kind="pair_far_control",
        refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, (far_r, far_r, 0, 0))]
    ))

    patterns.extend([
        Pattern(
            name="triple_line_same_ori",
            kind="triple_line",
            refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(0, e0), PlaqRef(0, (2, 0, 0, 0))]
        ),
        Pattern(
            name="triple_L_same_ori",
            kind="triple_L",
            refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(0, e0), PlaqRef(0, e1)]
        ),
        Pattern(
            name="triple_star_same_link",
            kind="triple_star",
            refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, (0, 0, 0, 0)), PlaqRef(2, (0, 0, 0, 0))]
        ),
        Pattern(
            name=f"triple_far_control_r{far_r}",
            kind="triple_far_control",
            refs=[
                PlaqRef(0, (0, 0, 0, 0)),
                PlaqRef(1, (far_r, 0, 0, 0)),
                PlaqRef(2, (0, far_r, 0, 0)),
```

```
            ]
        ),
    ])

    patterns.extend([
        Pattern(
            name="quad_line_same_ori",
            kind="quad_line",
            refs=[
                PlaqRef(0, (0, 0, 0, 0)),
                PlaqRef(0, e0),
                PlaqRef(0, (2, 0, 0, 0)),
                PlaqRef(0, (3, 0, 0, 0)),
            ]
        ),
        Pattern(
            name="quad_square_same_ori",
            kind="quad_square",
            refs=[
                PlaqRef(0, (0, 0, 0, 0)),
                PlaqRef(0, e0),
                PlaqRef(0, e1),
                PlaqRef(0, (1, 1, 0, 0)),
            ]
        ),
        Pattern(
            name="quad_local_mixed",
            kind="quad_local_mixed",
            refs=[
                PlaqRef(0, (0, 0, 0, 0)),
                PlaqRef(1, (0, 0, 0, 0)),
                PlaqRef(2, (0, 0, 0, 0)),
                PlaqRef(3, (0, 0, 0, 0)),
            ]
        ),
        Pattern(
            name=f"quad_far_control_r{far_r}",
            kind="quad_far_control",
            refs=[
                PlaqRef(0, (0, 0, 0, 0)),
                PlaqRef(1, (far_r, 0, 0, 0)),
                PlaqRef(2, (0, far_r, 0, 0)),
                PlaqRef(3, (0, 0, far_r, 0)),
            ]
        ),
    ])

    return patterns


def nonempty_submasks(k: int) -> List[int]:
    return list(range(1, 1 << k))


def partitions_of_mask(mask: int):
    """
    Generate set partitions of bitmask mask.
    Each partition is represented as a list of block masks.
    """
    if mask == 0:
        yield []
        return

    b = mask & -mask
    rest = mask ^ b

    for part in partitions_of_mask(rest):
        yield [b] + part

        for i in range(len(part)):
            new_part = part.copy()
            new_part[i] = new_part[i] | b
            yield new_part


def cumulant_from_moments(k: int, moments: Dict[int, float]) -> float:
    full = (1 << k) - 1
    total = 0.0
```

```
    for part in partitions_of_mask(full):
        r = len(part)
        coeff = ((-1) ** (r - 1)) * math.factorial(r - 1)
        prod = 1.0
        for block in part:
            prod *= moments[block]
        total += coeff * prod

    return float(total)


def pattern_subset_product_mean(X: torch.Tensor, pattern: Pattern, mask: int) -> float:
    """
    X shape: [6, L, L, L, L]
    Computes translation average of product over selected pattern refs.
    """
    prod = None

    for j, ref in enumerate(pattern.refs):
        if not (mask & (1 << j)):
            continue

        F = shift_field(X[ref.ori], ref.shift)
        prod = F if prod is None else prod * F

    return float(prod.mean().detach().cpu().item())


# ============================================================
# Moment accumulator
# ============================================================

class MomentAccumulator:
    def __init__(self, patterns: List[Pattern]):
        self.patterns = patterns
        self.sums = {}
        self.counts = {}
        self.q_sum = 0.0
        self.q_count = 0

        for pat in patterns:
            k = len(pat.refs)
            for mask in nonempty_submasks(k):
                self.sums[(pat.name, mask)] = 0.0
                self.counts[(pat.name, mask)] = 0

    def add_config(self, X: torch.Tensor):
        self.q_sum += float(X.mean().detach().cpu().item())
        self.q_count += 1

        for pat in self.patterns:
            k = len(pat.refs)
            for mask in nonempty_submasks(k):
                val = pattern_subset_product_mean(X, pat, mask)
                key = (pat.name, mask)
                self.sums[key] += val
                self.counts[key] += 1

    def finalize_rows(self, meta: Dict) -> List[Dict]:
        rows = []
        q_meas = self.q_sum / max(1, self.q_count)

        for pat in self.patterns:
            k = len(pat.refs)
            moments = {}

            for mask in nonempty_submasks(k):
                key = (pat.name, mask)
                moments[mask] = self.sums[key] / max(1, self.counts[key])

            kappa = cumulant_from_moments(k, moments)
            tau = mst_tau(pat.refs)
            full_mask = (1 << k) - 1
            full_moment = moments[full_mask]

            refs_json = json.dumps([
```

```
                {"ori": r.ori, "plane": PLAQUETTE_ORIS[r.ori], "shift": r.shift}
                for r in pat.refs
            ])

            row = dict(meta)
            row.update({
                "pattern_name": pat.name,
                "pattern_kind": pat.kind,
                "support_size": k,
                "tau_mst": tau,
                "refs_json": refs_json,
                "n_cfg": self.q_count,
                "q_meas_global": q_meas,
                "full_moment": full_moment,
                "connected_cumulant": kappa,
                "abs_cumulant": abs(kappa),
                "abs_cumulant_over_qk": abs(kappa) / ((q_meas + EPS) ** k),
                "abs_cumulant_over_q_rooted": abs(kappa) / (q_meas + EPS),
                "signed_cumulant_over_qk": kappa / ((q_meas + EPS) ** k),
                "moments_json": json.dumps({str(mask): moments[mask] for mask in sorted(moments)}),
            })

            rows.append(row)

        return rows


# ============================================================
# Run functions
# ============================================================

def field_cache_path(L: int, beta: float, start_mode: str) -> str:
    btag = str(beta).replace(".", "p")
    return os.path.join(FIELD_CACHE_DIR, f"thermalized_SU2_L{L}_beta{btag}_{start_mode}.pt")


def init_or_load_field(L: int, beta: float, start_mode: str):
    cache = field_cache_path(L, beta, start_mode)

    if LOAD_FIELD_CACHE and os.path.exists(cache):
        print(f"[cache] loading thermalized field: {cache}")
        U = torch.load(cache, map_location=DEVICE).to(device=DEVICE, dtype=DTYPE)
        return U, True

    print(f"[init] {start_mode} L={L} beta={beta}")

    if start_mode == "cold":
        U = cold_su2_links(L)
    elif start_mode == "hot":
        U = hot_su2_links(L)
    else:
        raise ValueError(f"Unknown START_MODE={start_mode}")

    return U, False


def thermalize(U: torch.Tensor, beta: float, parity_masks, sigma: float):
    logs = []
    t0 = time.time()

    for sweep in range(1, THERMAL_SWEEPS + 1):
        acc = metropolis_sweep(U, beta, sigma, parity_masks)

        if ADAPT_PROPOSAL and (sweep % ADAPT_EVERY == 0) and sweep <= max(50, THERMAL_SWEEPS // 2):
            sigma *= math.exp(ADAPT_RATE * (acc - ADAPT_TARGET_ACC))
            sigma = float(np.clip(sigma, 0.03, 1.50))

        if sweep == 1 or sweep % 25 == 0 or sweep == THERMAL_SWEEPS:
            print(f"  [therm] sweep={sweep:5d}/{THERMAL_SWEEPS} acc={acc:.4f} sigma={sigma:.5f}")

        logs.append({
            "sweep": sweep,
            "acceptance": acc,
            "proposal_sigma": sigma,
            "elapsed_s": time.time() - t0,
        })

    return U, sigma, logs
```

```
    [redacted]


def collect_calibration_samples(U: torch.Tensor, beta: float, parity_masks, sigma: float) -> np.ndarray:
    samples = []

    print(f"[calib] collecting {N_CALIB_CFG} calibration configs")

    for ci in range(1, N_CALIB_CFG + 1):
        for _ in range(BETWEEN_SWEEPS):
            metropolis_sweep(U, beta, sigma, parity_masks)

        phi = plaquette_phi(U)
        flat = phi.flatten()
        n = flat.numel()
        take = min(CALIB_MAX_PHI_PER_CFG, n)
        idx = torch.randint(0, n, (take,), device=DEVICE)
        vals = flat[idx].detach().cpu().numpy()
        samples.append(vals)

        print(f"  [calib] cfg={ci:4d}/{N_CALIB_CFG}, sampled={take}")

    return np.concatenate(samples, axis=0)


def run_one_combo(L: int, beta: float):
    print("=" * 100)
    print(f"[combo] L={L}, beta={beta}, mode={RUN_MODE}")
    print("=" * 100)

    parity_masks = [make_parity_mask(L, 0), make_parity_mask(L, 1)]

    sigma = BASE_PROPOSAL_SIGMA
    if sigma is None:
        sigma = 0.80 / math.sqrt(beta)

    U, loaded = init_or_load_field(L, beta, START_MODE)

    thermal_logs = []
    if not loaded:
        U, sigma, thermal_logs = thermalize(U, beta, parity_masks, sigma)

        if SAVE_FIELD_CACHE:
            cache = field_cache_path(L, beta, START_MODE)
            torch.save(U.detach().cpu(), cache)
            print(f"[cache] saved thermalized field: {cache}")
    else:
        print("[therm] skipped; using cached field")

    phi_samples = collect_calibration_samples(U, beta, parity_masks, sigma)

    threshold_rows = []
    thresholds = {}

    for eta in ETA_LIST:
        for qtar in Q_TARGET_LIST:
            t = calibrate_threshold_from_samples(phi_samples, eta, qtar)
            thresholds[(eta, qtar)] = t
            achieved = float(sigmoid_np((phi_samples - t) / eta).mean())

            threshold_rows.append({
                "L": L,
                "beta": beta,
                "eta": eta,
                "q_target": qtar,
                "threshold_t": t,
                "q_achieved_on_calib_samples": achieved,
                "n_phi_calib_samples": len(phi_samples),
            })

            print(f"  [threshold] eta={eta:.5g} qtar={qtar:.5g} t={t:.8f} q_calib={achieved:.8g}")

    patterns = build_patterns(L)
    print(f"[patterns] {len(patterns)} support patterns")

    accumulators = {
        (eta, qtar): MomentAccumulator(patterns)
        for eta in ETA_LIST
```

```
        for qtar in Q_TARGET_LIST
    }

    meas_logs = []
    t0 = time.time()

    print(f"[meas] collecting {N_MEAS_CFG} measurement configs")

    for mi in range(1, N_MEAS_CFG + 1):
        accs = []

        for _ in range(BETWEEN_SWEEPS):
            accs.append(metropolis_sweep(U, beta, sigma, parity_masks))

        phi = plaquette_phi(U)

        for eta in ETA_LIST:
            for qtar in Q_TARGET_LIST:
                t = thresholds[(eta, qtar)]
                X = smooth_X_from_phi(phi, t, eta)
                accumulators[(eta, qtar)].add_config(X)

        mean_acc = float(np.mean(accs))

        if mi == 1 or mi % 10 == 0 or mi == N_MEAS_CFG:
            print(f"  [meas] cfg={mi:5d}/{N_MEAS_CFG} acc={mean_acc:.4f} elapsed={time.time()-t0:.1f}s")

        meas_logs.append({
            "L": L,
            "beta": beta,
            "meas_cfg": mi,
            "acceptance_mean_between": mean_acc,
            "proposal_sigma": sigma,
            "elapsed_s": time.time() - t0,
        })

    if SAVE_FIELD_CACHE:
        cache = field_cache_path(L, beta, START_MODE)
        torch.save(U.detach().cpu(), cache)
        print(f"[cache] updated field: {cache}")

    cumulant_rows = []

    for eta in ETA_LIST:
        for qtar in Q_TARGET_LIST:
            meta = {
                "run_id": RUN_ID,
                "run_mode": RUN_MODE,
                "L": L,
                "beta": beta,
                "start_mode": START_MODE,
                "eta": eta,
                "q_target": qtar,
                "threshold_t": thresholds[(eta, qtar)],
                "thermal_sweeps": THERMAL_SWEEPS,
                "between_sweeps": BETWEEN_SWEEPS,
                "n_calib_cfg": N_CALIB_CFG,
                "n_meas_cfg": N_MEAS_CFG,
            }

            cumulant_rows.extend(accumulators[(eta, qtar)].finalize_rows(meta))

    return threshold_rows, cumulant_rows, thermal_logs, meas_logs


# ============================================================
# Output / reporting
# ============================================================

def make_summary(cumulants_df: pd.DataFrame) -> pd.DataFrame:
    if cumulants_df.empty:
        return pd.DataFrame()

    group_cols = [
        "L", "beta", "eta", "q_target", "support_size", "pattern_kind"
    ]

    rows = []
```

```
    rows = []

    for keys, g in cumulants_df.groupby(group_cols):
        d = dict(zip(group_cols, keys))
        d.update({
            "n_patterns": len(g),
            "max_abs_cumulant_over_qk": float(g["abs_cumulant_over_qk"].max()),
            "mean_abs_cumulant_over_qk": float(g["abs_cumulant_over_qk"].mean()),
            "max_abs_cumulant_over_q_rooted": float(g["abs_cumulant_over_q_rooted"].max()),
            "mean_abs_cumulant_over_q_rooted": float(g["abs_cumulant_over_q_rooted"].mean()),
            "max_abs_cumulant": float(g["abs_cumulant"].max()),
            "mean_abs_cumulant": float(g["abs_cumulant"].mean()),
            "mean_q_meas_global": float(g["q_meas_global"].mean()),
        })
        rows.append(d)

    return pd.DataFrame(rows)


def make_pair_decay(cumulants_df: pd.DataFrame) -> pd.DataFrame:
    rows = []

    if cumulants_df.empty:
        return pd.DataFrame()

    pair_df = cumulants_df[cumulants_df["support_size"] == 2].copy()
    if pair_df.empty:
        return pd.DataFrame()

    group_cols = ["L", "beta", "eta", "q_target"]

    for keys, g in pair_df.groupby(group_cols):
        d = dict(zip(group_cols, keys))

        gg = g[g["pattern_name"].str.contains("pair_same_ori_axis_r", regex=False)].copy()
        if len(gg) < 2:
            continue

        rs = []
        ys = []

        for _, row in gg.iterrows():
            name = row["pattern_name"]
            try:
                r = int(name.split("_r")[-1])
            except Exception:
                continue

            y = float(row["abs_cumulant_over_qk"])
            if y > 0:
                rs.append(r)
                ys.append(y)

        if len(rs) >= 2:
            x = np.asarray(rs, dtype=np.float64)
            ly = np.log(np.asarray(ys, dtype=np.float64))
            slope, intercept = np.polyfit(x, ly, 1)

            d.update({
                "n_points": len(rs),
                "exp_decay_slope_vs_r": float(slope),
                "exp_decay_intercept": float(intercept),
                "decay_rate_positive": float(-slope),
            })
            rows.append(d)

    return pd.DataFrame(rows)


def write_outputs(all_thresholds, all_cumulants, all_thermal_logs, all_meas_logs):
    thresholds_df = pd.DataFrame(all_thresholds)
    cumulants_df = pd.DataFrame(all_cumulants)
    thermal_df = pd.DataFrame(all_thermal_logs)
    meas_df = pd.DataFrame(all_meas_logs)

    summary_df = make_summary(cumulants_df)
    pair_decay_df = make_pair_decay(cumulants_df)
```

```
thresholds_csv = os.path.join(OUTDIR, "thresholds.csv")
cumulants_csv = os.path.join(OUTDIR, "connected_cumulants.csv")
thermal_csv = os.path.join(OUTDIR, "thermalization_logs.csv")
meas_csv = os.path.join(OUTDIR, "measurement_logs.csv")
summary_csv = os.path.join(OUTDIR, "summary_by_pattern_kind.csv")
pair_decay_csv = os.path.join(OUTDIR, "pair_decay_fits.csv")
metadata_json = os.path.join(OUTDIR, "metadata.json")
readout_md = os.path.join(OUTDIR, "READOUT.md")

thresholds_df.to_csv(thresholds_csv, index=False)
cumulants_df.to_csv(cumulants_csv, index=False)
thermal_df.to_csv(thermal_csv, index=False)
meas_df.to_csv(meas_csv, index=False)
summary_df.to_csv(summary_csv, index=False)
pair_decay_df.to_csv(pair_decay_csv, index=False)

metadata = {
    "run_id": RUN_ID,
    "run_tag": RUN_TAG,
    "run_mode": RUN_MODE,
    "seed": SEED,
    "device": str(DEVICE),
    "cuda_device_name": torch.cuda.get_device_name(0) if torch.cuda.is_available() else None,
    "L_LIST": L_LIST,
    "BETA_LIST": BETA_LIST,
    "START_MODE": START_MODE,
    "THERMAL_SWEEPS": THERMAL_SWEEPS,
    "N_CALIB_CFG": N_CALIB_CFG,
    "N_MEAS_CFG": N_MEAS_CFG,
    "BETWEEN_SWEEPS": BETWEEN_SWEEPS,
    "ETA_LIST": ETA_LIST,
    "Q_TARGET_LIST": Q_TARGET_LIST,
    "R_DISTANCES": R_DISTANCES,
    "BASE_PROPOSAL_SIGMA": BASE_PROPOSAL_SIGMA,
    "ADAPT_PROPOSAL": ADAPT_PROPOSAL,
    "ADAPT_TARGET_ACC": ADAPT_TARGET_ACC,
    "CALIB_MAX_PHI_PER_CFG": CALIB_MAX_PHI_PER_CFG,
    "outdir": OUTDIR,
    "field_cache_dir": FIELD_CACHE_DIR,
    "purpose": "Finite-volume numerical diagnostic for Balaban smooth-source expansion via connected cumulants.",
    "does_not_prove": [
        "analytic BS cluster expansion",
        "uniform-in-L convergence",
        "uniform-in-eta hard-threshold bridge",
        "top-p de-Poissonization",
        "Yang-Mills mass gap",
    ],
}

with open(metadata_json, "w") as f:
    json.dump(metadata, f, indent=2)

lines = []
lines.append(f"# {RUN_ID}")
lines.append("")
lines.append("## Purpose")
lines.append("")
lines.append("Numerical diagnostic for BS = Balaban smooth-source expansion.")
lines.append("")
lines.append("The measured quantity is the connected square-free cumulant of smoothed plaquette defects.")
lines.append("")
lines.append("This does not prove BS; it tests whether the expected coefficient hierarchy is numerically plausible.")
lines.append("")
lines.append("## Files")
lines.append("")
lines.append("- `thresholds.csv`: calibrated smooth thresholds.")
lines.append("- `connected_cumulants.csv`: main connected coefficient table.")
lines.append("- `summary_by_pattern_kind.csv`: aggregate summary.")
lines.append("- `pair_decay_fits.csv`: exponential decay fits for same-orientation axis pairs.")
lines.append("- `thermalization_logs.csv`: thermalization acceptance logs.")
lines.append("- `measurement_logs.csv`: measurement acceptance logs.")
lines.append("- `metadata.json`: run metadata.")
lines.append("")
lines.append("## Main interpretation checks")
lines.append("")
lines.append("1. `abs_cumulant_over_qk` should not grow badly with L.")
lines.append("2. Pair coefficients should decay with plaquette separation.")
```

```
    lines.append("3. Far controls should be small relative to local connected supports.")
    lines.append("4. Smaller eta should not cause uncontrolled coefficient blow-up.")
    lines.append("5. Rooted scale `abs_cumulant_over_q_rooted` is the crude numerical proxy for pinned source-polymer burden.")
    lines.append("")

    if not summary_df.empty:
        lines.append("## Worst aggregate rows")
        lines.append("")
        worst = summary_df.sort_values("max_abs_cumulant_over_qk", ascending=False).head(20)
        lines.append(worst.to_markdown(index=False))
        lines.append("")

    if not pair_decay_df.empty:
        lines.append("## Pair decay fits")
        lines.append("")
        lines.append(pair_decay_df.to_markdown(index=False))
        lines.append("")

    with open(readout_md, "w") as f:
        f.write("\n".join(lines))

    zip_path = os.path.join(BASE_OUTDIR, f"{RUN_ID}.zip")
    with zipfile.ZipFile(zip_path, "w", compression=zipfile.ZIP_DEFLATED) as zf:
        for root, _, files in os.walk(OUTDIR):
            for fn in files:
                path = os.path.join(root, fn)
                arc = os.path.relpath(path, OUTDIR)
                zf.write(path, arcname=arc)

    print("=" * 100)
    print("[outputs written]")
    print(f"OUTDIR: {OUTDIR}")
    print(f"ZIP:    {zip_path}")
    print("=" * 100)

    return {
        "thresholds_csv": thresholds_csv,
        "cumulants_csv": cumulants_csv,
        "thermal_csv": thermal_csv,
        "meas_csv": meas_csv,
        "summary_csv": summary_csv,
        "pair_decay_csv": pair_decay_csv,
        "metadata_json": metadata_json,
        "readout_md": readout_md,
        "zip_path": zip_path,
    }


# ============================================================
# Main
# ============================================================

def main():
    all_thresholds = []
    all_cumulants = []
    all_thermal_logs = []
    all_meas_logs = []

    t_all = time.time()

    for L in L_LIST:
        for beta in BETA_LIST:
            threshold_rows, cumulant_rows, thermal_logs, meas_logs = run_one_combo(L, beta)

            for r in threshold_rows:
                r["run_id"] = RUN_ID
                r["run_mode"] = RUN_MODE

            for r in thermal_logs:
                r["run_id"] = RUN_ID
                r["run_mode"] = RUN_MODE
                r["L"] = L
                r["beta"] = beta

            for r in meas_logs:
                r["run_id"] = RUN_ID
                r["run_mode"] = RUN_MODE
```

```
            all_thresholds.extend(threshold_rows)
            all_cumulants.extend(cumulant_rows)
            all_thermal_logs.extend(thermal_logs)
            all_meas_logs.extend(meas_logs)

            write_outputs(all_thresholds, all_cumulants, all_thermal_logs, all_meas_logs)

    paths = write_outputs(all_thresholds, all_cumulants, all_thermal_logs, all_meas_logs)

    print(f"[done] elapsed_total_s={time.time()-t_all:.2f}")
    print("[important]")
    print("Upload the ZIP and READOUT.md for review.")
    print(paths["zip_path"])


if __name__ == "__main__":
    main()
```

```
[device] CUDA: NVIDIA A100-SXM4-40GB
[run] PMBSF_v17_BS_smooth_source_connected_cumulants_20260524_174456
[outdir] /content/PMBSF_v17_BS_output/PMBSF_v17_BS_smooth_source_connected_cumulants_20260524_174456
================================================================================================
[combo] L=12, beta=3.5, mode=pilot
================================================================================================
[init] hot L=12 beta=3.5
  [therm] sweep=    1/100 acc=0.6242 sigma=0.42762
  [therm] sweep=   25/100 acc=0.3271 sigma=0.41468
  [therm] sweep=   50/100 acc=0.3176 sigma=0.39203
  [therm] sweep=   75/100 acc=0.3163 sigma=0.39203
  [therm] sweep=  100/100 acc=0.3125 sigma=0.39203
[cache] saved thermalized field: /content/PMBSF_v17_thermalized_fields/thermalized_SU2_L12_beta3p5_hot.pt
[calib] collecting 8 calibration configs
  [calib] cfg=   1/8, sampled=124416
  [calib] cfg=   2/8, sampled=124416
  [calib] cfg=   3/8, sampled=124416
  [calib] cfg=   4/8, sampled=124416
  [calib] cfg=   5/8, sampled=124416
  [calib] cfg=   6/8, sampled=124416
  [calib] cfg=   7/8, sampled=124416
  [calib] cfg=   8/8, sampled=124416
  [threshold] eta=0.05 qtar=0.003 t=1.06241488 q_calib=0.0029999993
  [threshold] eta=0.05 qtar=0.01 t=0.88518872 q_calib=0.010000004
  [threshold] eta=0.1 qtar=0.003 t=1.15539970 q_calib=0.003000001
  [threshold] eta=0.1 qtar=0.01 t=0.97037969 q_calib=0.009999997
[patterns] 20 support patterns
[meas] collecting 30 measurement configs
  [meas] cfg=    1/30 acc=0.3118 elapsed=0.7s
  [meas] cfg=   10/30 acc=0.3111 elapsed=6.8s
  [meas] cfg=   20/30 acc=0.3102 elapsed=13.7s
  [meas] cfg=   30/30 acc=0.3105 elapsed=20.6s
[cache] updated field: /content/PMBSF_v17_thermalized_fields/thermalized_SU2_L12_beta3p5_hot.pt
================================================================================================
[outputs written]
OUTDIR: /content/PMBSF_v17_BS_output/PMBSF_v17_BS_smooth_source_connected_cumulants_20260524_174456
ZIP:    /content/PMBSF_v17_BS_output/PMBSF_v17_BS_smooth_source_connected_cumulants_20260524_174456.zip
================================================================================================
================================================================================================
[outputs written]
OUTDIR: /content/PMBSF_v17_BS_output/PMBSF_v17_BS_smooth_source_connected_cumulants_20260524_174456
ZIP:    /content/PMBSF_v17_BS_output/PMBSF_v17_BS_smooth_source_connected_cumulants_20260524_174456.zip
================================================================================================
[done] elapsed_total_s=36.63
[important]
Upload the ZIP and READOUT.md for review.
/content/PMBSF_v17_BS_output/PMBSF_v17_BS_smooth_source_connected_cumulants_20260524_174456.zip
```

```
# ============================================================
# PMBSF_v17b_BS_smooth_source_connected_cumulants_GOOD.py
#
# Purpose:
#   Production numerical diagnostic for BS = Balaban smooth-source expansion.
#
#   Tests the finite-volume numerical shadow of:
#
#       log E_W prod_p (1 + u_p X_{p,eta})
#       = sum connected source polymers
#
#   by estimating connected square-free cumulants of smoothed
#   high-plaquette observables:
#
```

```
#       X_{p,eta} = sigmoid((phi(U_p)-t)/eta)
#
#   where
#
#       phi(U_p) = 1 - (1/2) ReTr(U_p) = 1 - q0(U_p).
#
#   Main diagnostic:
#
#       |kappa(B)| / q_eta^{|B|}
#
#   versus support size, support geometry, and tree distance tau(B).
#
#   This does NOT prove BS. It tests whether the BS coefficient hierarchy
#   is numerically plausible for smoothed Wilson plaquette defects.
#
# Recommended:
#   A100 GPU.
#
# Outputs:
#   thresholds.csv
#   connected_cumulants.csv
#   summary_by_pattern_kind.csv
#   pair_decay_fits.csv
#   block_jackknife_diagnostics.csv
#   thermalization_logs.csv
#   measurement_logs.csv
#   metadata.json
#   READOUT.md
#   ZIP archive
# ============================================================

import os
import math
import json
import time
import zipfile
import random
import warnings
from dataclasses import dataclass
from typing import Dict, List, Tuple, Any

import numpy as np
import pandas as pd
import torch


warnings.filterwarnings("ignore", category=FutureWarning)


# ============================================================
# Configuration
# ============================================================

RUN_TAG = "PMBSF_v17b_BS_smooth_source_connected_cumulants_GOOD"

# Main mode.
# Use "smoke" for a 2-3 minute check.
# Use "good_a100" for a real run.
RUN_MODE = "good_a100"  # "smoke" or "good_a100"

# Force a new thermalization in this clean v17b cache namespace.
FORCE_RETHERMALIZE = False

# Hot is enough for the first production pass.
# Turn on cold as an audit after hot passes.
START_MODES = ["hot"]
# START_MODES = ["hot", "cold"]

# Beta set.
# beta=3.5 is the stress point. beta=4.0 is the sanity comparison.
BETA_LIST = [3.5, 4.0]

# Lattice list.
L_LIST = [12, 16, 24]

# Smooth widths and target densities.
ETA_LIST = [0.025, 0.05, 0.10]
```

```
Q_TARGET_LIST = [0.001, 0.003, 0.01]

# Pair distances.
R_DISTANCES_BY_L = {
    12: [1, 2, 3, 4],
    16: [1, 2, 3, 4, 6],
    24: [1, 2, 3, 4, 6, 8],
}

# Per-L production budgets.
# Runtime is dominated by sweeps at L=24.
GOOD_L_SETTINGS = {
    12: {
        "thermal_sweeps": 600,
        "n_calib_cfg": 24,
        "n_meas_cfg": 420,
        "between_sweeps": 12,
        "calib_max_phi_per_cfg": 160_000,
        "block_size": 20,
    },
    16: {
        "thermal_sweeps": 700,
        "n_calib_cfg": 24,
        "n_meas_cfg": 320,
        "between_sweeps": 12,
        "calib_max_phi_per_cfg": 220_000,
        "block_size": 20,
    },
    24: {
        "thermal_sweeps": 800,
        "n_calib_cfg": 20,
        "n_meas_cfg": 220,
        "between_sweeps": 10,
        "calib_max_phi_per_cfg": 300_000,
        "block_size": 20,
    },
}

SMOKE_L_SETTINGS = {
    12: {
        "thermal_sweeps": 120,
        "n_calib_cfg": 8,
        "n_meas_cfg": 40,
        "between_sweeps": 8,
        "calib_max_phi_per_cfg": 120_000,
        "block_size": 10,
    }
}

if RUN_MODE == "smoke":
    L_LIST = [12]
    BETA_LIST = [3.5]
    ETA_LIST = [0.05, 0.10]
    Q_TARGET_LIST = [0.003, 0.01]
    L_SETTINGS = SMOKE_L_SETTINGS
else:
    L_SETTINGS = GOOD_L_SETTINGS

# Proposal tuning.
# Prior pilot had acceptance ~0.31 with sigma=0.80/sqrt(beta), too aggressive.
# This starts gentler and adapts during thermalization.
BASE_PROPOSAL_SIGMA = None  # if None: 0.50 / sqrt(beta)

ADAPT_PROPOSAL = True
ADAPT_EVERY = 10
ADAPT_TARGET_ACC = 0.50
ADAPT_RATE = 0.12
ADAPT_UNTIL_FRACTION_OF_THERM = 0.75

# Random seed.
SEED = 23061721

# Output/cache.
BASE_OUTDIR = "/content/PMBSF_v17b_BS_output"
FIELD_CACHE_DIR = "/content/PMBSF_v17b_thermalized_fields"
SAVE_FIELD_CACHE = True
LOAD_FIELD_CACHE = True
```

```
# Precision.
DTYPE = torch.float32
SIGMOID_CLIP = 60.0
EPS = 1e-30


# ============================================================
# Device setup
# ============================================================

def get_device():
    if torch.cuda.is_available():
        dev = torch.device("cuda")
        print(f"[device] CUDA: {torch.cuda.get_device_name(0)}")
        return dev
    print("[device] CPU fallback")
    return torch.device("cpu")


DEVICE = get_device()
torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)

os.makedirs(BASE_OUTDIR, exist_ok=True)
os.makedirs(FIELD_CACHE_DIR, exist_ok=True)

RUN_ID = f"{RUN_TAG}_{time.strftime('%Y%m%d_%H%M%S')}"
OUTDIR = os.path.join(BASE_OUTDIR, RUN_ID)
os.makedirs(OUTDIR, exist_ok=True)

print(f"[run] {RUN_ID}")
print(f"[outdir] {OUTDIR}")


# ============================================================
# SU(2) quaternion algebra
# q = (q0, q1, q2, q3)
# ============================================================

def qmul(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    a0, a1, a2, a3 = a.unbind(dim=-1)
    b0, b1, b2, b3 = b.unbind(dim=-1)
    return torch.stack(
        (
            a0*b0 - a1*b1 - a2*b2 - a3*b3,
            a0*b1 + a1*b0 + a2*b3 - a3*b2,
            a0*b2 - a1*b3 + a2*b0 + a3*b1,
            a0*b3 + a1*b2 - a2*b1 + a3*b0,
        ),
        dim=-1,
    )


def qconj(a: torch.Tensor) -> torch.Tensor:
    out = a.clone()
    out[..., 1:] *= -1
    return out


def qnorm(a: torch.Tensor) -> torch.Tensor:
    return torch.sqrt(torch.sum(a * a, dim=-1, keepdim=True).clamp_min(EPS))


def qnormalize(a: torch.Tensor) -> torch.Tensor:
    return a / qnorm(a)


def random_su2(shape, device=DEVICE, dtype=DTYPE) -> torch.Tensor:
    q = torch.randn((*shape, 4), device=device, dtype=dtype)
    return qnormalize(q)


def cold_su2_links(L: int) -> torch.Tensor:
    U = torch.zeros((4, L, L, L, L, 4), device=DEVICE, dtype=DTYPE)
    U[..., 0] = 1.0
```

```
    return U


def hot_su2_links(L: int) -> torch.Tensor:
    return random_su2((4, L, L, L, L), device=DEVICE, dtype=DTYPE)


def random_near_identity(shape, sigma: float) -> torch.Tensor:
    v = sigma * torch.randn((*shape, 3), device=DEVICE, dtype=DTYPE)
    q = torch.cat(
        [torch.ones((*shape, 1), device=DEVICE, dtype=DTYPE), v],
        dim=-1,
    )
    return qnormalize(q)


def shift_field(F: torch.Tensor, shift: Tuple[int, int, int, int]) -> torch.Tensor:
    """
    Return F(x + shift), aligned at x.
    """
    return torch.roll(F, shifts=tuple(-s for s in shift), dims=(0, 1, 2, 3))


def unit_shift(mu: int) -> Tuple[int, int, int, int]:
    s = [0, 0, 0, 0]
    s[mu] = 1
    return tuple(s)


def add_shift(a, b):
    return tuple(int(a[i] + b[i]) for i in range(4))


# ============================================================
# Gauge action / Metropolis update
# ============================================================

PLAQUETTE_ORIS = [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]


def staple_for_link(U: torch.Tensor, mu: int) -> torch.Tensor:
    """
    Staple V_mu(x) for local Wilson action contribution:

        - beta * scalar_part(U_mu(x) * V_mu(x))

    This is the same convention as earlier PMBSF Wilson runs.
    """
    L = U.shape[1]
    V = torch.zeros((L, L, L, L, 4), device=U.device, dtype=U.dtype)
    emu = unit_shift(mu)

    for nu in range(4):
        if nu == mu:
            continue

        enu = unit_shift(nu)

        U_nu_xpmu = shift_field(U[nu], emu)
        U_mu_xpnu = shift_field(U[mu], enu)
        U_nu_x = U[nu]

        # Positive staple:
        # U_nu(x+mu) U_mu^\dagger(x+nu) U_nu^\dagger(x)
        V_pos = qmul(qmul(U_nu_xpmu, qconj(U_mu_xpnu)), qconj(U_nu_x))

        # Negative staple:
        # U_nu^\dagger(x-nu+mu) U_mu^\dagger(x-nu) U_nu(x-nu)
        minus_enu = tuple(-x for x in enu)
        shift_munu = add_shift(emu, minus_enu)

        U_nu_xmnu_pmu = shift_field(U[nu], shift_munu)
        U_mu_xmnu = shift_field(U[mu], minus_enu)
        U_nu_xmnu = shift_field(U[nu], minus_enu)

        V_neg = qmul(qmul(qconj(U_nu_xmnu_pmu), qconj(U_mu_xmnu)), U_nu_xmnu)
```

```
            V = V + V_pos + V_neg

        return V


    def make_parity_mask(L: int, parity: int) -> torch.Tensor:
        coords = torch.meshgrid(
            torch.arange(L, device=DEVICE),
            torch.arange(L, device=DEVICE),
            torch.arange(L, device=DEVICE),
            torch.arange(L, device=DEVICE),
            indexing="ij",
        )
        s = coords[0] + coords[1] + coords[2] + coords[3]
        return (s % 2) == parity


    def metropolis_sweep(U: torch.Tensor, beta: float, sigma: float, parity_masks) -> float:
        accs = []

        for mu in range(4):
            for parity in (0, 1):
                mask = parity_masks[parity]

                V = staple_for_link(U, mu)
                old_score = qmul(U[mu], V)[..., 0]

                R = random_near_identity(U[mu].shape[:-1], sigma)
                U_prop = qnormalize(qmul(R, U[mu]))
                new_score = qmul(U_prop, V)[..., 0]

                log_accept_ratio = beta * (new_score - old_score)
                logu = torch.log(torch.rand_like(log_accept_ratio).clamp_min(EPS))
                accept = (logu < log_accept_ratio) & mask

                U[mu] = torch.where(accept[..., None], U_prop, U[mu])

                if mask.any():
                    accs.append(accept[mask].float().mean().item())

        return float(np.mean(accs))


    def plaquette_phi(U: torch.Tensor) -> torch.Tensor:
        """
        Return phi for all six plaquette orientations.

        Shape:
            [6, L, L, L, L]

        phi:
            phi = 1 - scalar_part(U_p)
        """
        phis = []

        for (mu, nu) in PLAQUETTE_ORIS:
            emu = unit_shift(mu)
            enu = unit_shift(nu)

            U_mu_x = U[mu]
            U_nu_xpmu = shift_field(U[nu], emu)
            U_mu_xpnu = shift_field(U[mu], enu)
            U_nu_x = U[nu]

            P = qmul(
                qmul(qmul(U_mu_x, U_nu_xpmu), qconj(U_mu_xpnu)),
                qconj(U_nu_x),
            )
            q0 = P[..., 0].clamp(-1.0, 1.0)
            phi = (1.0 - q0).clamp(0.0, 2.0)
            phis.append(phi)

        return torch.stack(phis, dim=0)


    # ============================================================
    # Smooth threshold calibration
```

```
# ============================================================

def sigmoid_np(z):
    z = np.clip(z, -SIGMOID_CLIP, SIGMOID_CLIP)
    return 1.0 / (1.0 + np.exp(-z))


def calibrate_threshold_from_samples(phi_samples: np.ndarray, eta: float, q_target: float) -> float:
    """
    Find t such that mean sigmoid((phi-t)/eta) = q_target.
    """
    samples = phi_samples.astype(np.float64)

    lo = float(samples.min() - 25.0 * eta)
    hi = float(samples.max() + 25.0 * eta)

    for _ in range(100):
        mid = 0.5 * (lo + hi)
        m = sigmoid_np((samples - mid) / eta).mean()
        if m > q_target:
            lo = mid
        else:
            hi = mid

    return float(0.5 * (lo + hi))


def smooth_X_from_phi(phi: torch.Tensor, threshold: float, eta: float) -> torch.Tensor:
    z = ((phi - threshold) / eta).clamp(-SIGMOID_CLIP, SIGMOID_CLIP)
    return torch.sigmoid(z)


# ============================================================
# Pattern / cumulant machinery
# ============================================================

@dataclass(frozen=True)
class PlaqRef:
    ori: int
    shift: Tuple[int, int, int, int]


@dataclass
class Pattern:
    name: str
    refs: List[PlaqRef]
    kind: str


def pair_dist(a: PlaqRef, b: PlaqRef) -> int:
    # Plaquette graph proxy: lattice L1 shift plus orientation mismatch.
    d = sum(abs(a.shift[i] - b.shift[i]) for i in range(4))
    if a.ori != b.ori:
        d += 1
    return int(d)


def mst_tau(refs: List[PlaqRef]) -> int:
    n = len(refs)
    if n <= 1:
        return 0

    used = [False] * n
    used[0] = True
    total = 0

    for _ in range(n - 1):
        best = None
        for i in range(n):
            if not used[i]:
                continue
            for j in range(n):
                if used[j]:
                    continue
                d = pair_dist(refs[i], refs[j])
                if best is None or d < best[0]:
                    best = (d, j)
```

```
            total += best[0]
            used[best[1]] = True

        return int(total)


    def build_patterns(L: int, r_distances: List[int]) -> List[Pattern]:
        e0 = (1, 0, 0, 0)
        e1 = (0, 1, 0, 0)
        e2 = (0, 0, 1, 0)

        patterns: List[Pattern] = []

        # Same-orientation pair families.
        for r in r_distances:
            if r < L // 2:
                patterns.append(Pattern(
                    name=f"pair_same_ori_axis_r{r}",
                    kind="pair_same_ori_axis",
                    refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(0, (r, 0, 0, 0))]
                ))
                patterns.append(Pattern(
                    name=f"pair_same_ori_diag_r{r}",
                    kind="pair_same_ori_diag",
                    refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(0, (r, r, 0, 0))]
                ))
                patterns.append(Pattern(
                    name=f"pair_mixed_axis_r{r}",
                    kind="pair_mixed_axis",
                    refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, (r, 0, 0, 0))]
                ))

        # Local pair controls.
        patterns.extend([
            Pattern(
                name="pair_incident_same_site_01_02",
                kind="pair_incident",
                refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, (0, 0, 0, 0))]
            ),
            Pattern(
                name="pair_incident_same_site_01_03",
                kind="pair_incident",
                refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(2, (0, 0, 0, 0))]
            ),
            Pattern(
                name="pair_near_shifted_mixed",
                kind="pair_mixed_near",
                refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, (1, 0, 0, 0))]
            ),
        ])

        far_r = max(2, L // 3)

        patterns.append(Pattern(
            name=f"pair_far_control_r{far_r}",
            kind="pair_far_control",
            refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, (far_r, far_r, 0, 0))]
        ))

        # Triples.
        patterns.extend([
            Pattern(
                name="triple_line_same_ori",
                kind="triple_line",
                refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(0, e0), PlaqRef(0, (2, 0, 0, 0))]
            ),
            Pattern(
                name="triple_L_same_ori",
                kind="triple_L",
                refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(0, e0), PlaqRef(0, e1)]
            ),
            Pattern(
                name="triple_star_same_link",
                kind="triple_star",
                refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, (0, 0, 0, 0)), PlaqRef(2, (0, 0, 0, 0))]
            ),
            Pattern(
```

```
                name="triple_mixed_chain",
                kind="triple_mixed_chain",
                refs=[PlaqRef(0, (0, 0, 0, 0)), PlaqRef(1, e0), PlaqRef(2, (2, 0, 0, 0))]
            ),
            Pattern(
                name=f"triple_far_control_r{far_r}",
                kind="triple_far_control",
                refs=[
                    PlaqRef(0, (0, 0, 0, 0)),
                    PlaqRef(1, (far_r, 0, 0, 0)),
                    PlaqRef(2, (0, far_r, 0, 0)),
                ]
            ),
        ])

        # Quadruples.
        patterns.extend([
            Pattern(
                name="quad_line_same_ori",
                kind="quad_line",
                refs=[
                    PlaqRef(0, (0, 0, 0, 0)),
                    PlaqRef(0, e0),
                    PlaqRef(0, (2, 0, 0, 0)),
                    PlaqRef(0, (3, 0, 0, 0)),
                ]
            ),
            Pattern(
                name="quad_square_same_ori",
                kind="quad_square",
                refs=[
                    PlaqRef(0, (0, 0, 0, 0)),
                    PlaqRef(0, e0),
                    PlaqRef(0, e1),
                    PlaqRef(0, (1, 1, 0, 0)),
                ]
            ),
            Pattern(
                name="quad_local_mixed",
                kind="quad_local_mixed",
                refs=[
                    PlaqRef(0, (0, 0, 0, 0)),
                    PlaqRef(1, (0, 0, 0, 0)),
                    PlaqRef(2, (0, 0, 0, 0)),
                    PlaqRef(3, (0, 0, 0, 0)),
                ]
            ),
            Pattern(
                name="quad_tree_mixed",
                kind="quad_tree_mixed",
                refs=[
                    PlaqRef(0, (0, 0, 0, 0)),
                    PlaqRef(1, e0),
                    PlaqRef(2, e1),
                    PlaqRef(3, e2),
                ]
            ),
            Pattern(
                name=f"quad_far_control_r{far_r}",
                kind="quad_far_control",
                refs=[
                    PlaqRef(0, (0, 0, 0, 0)),
                    PlaqRef(1, (far_r, 0, 0, 0)),
                    PlaqRef(2, (0, far_r, 0, 0)),
                    PlaqRef(3, (0, 0, far_r, 0)),
                ]
            ),
        ])

        return patterns


    def nonempty_submasks(k: int) -> List[int]:
        return list(range(1, 1 << k))


    def partitions_of_mask(mask: int):
```

```
    """
    Generate set partitions of bitmask mask.
    Each partition is represented as a list of block masks.
    """
    if mask == 0:
        yield []
        return

    b = mask & -mask
    rest = mask ^ b

    for part in partitions_of_mask(rest):
        yield [b] + part

        for i in range(len(part)):
            new_part = part.copy()
            new_part[i] = new_part[i] | b
            yield new_part


def cumulant_from_moments(k: int, moments: Dict[int, float]) -> float:
    full = (1 << k) - 1
    total = 0.0

    for part in partitions_of_mask(full):
        r = len(part)
        coeff = ((-1) ** (r - 1)) * math.factorial(r - 1)
        prod = 1.0
        for block in part:
            prod *= moments[block]
        total += coeff * prod

    return float(total)


def pattern_subset_product_mean(X: torch.Tensor, pattern: Pattern, mask: int) -> float:
    """
    X shape: [6, L, L, L, L].
    Computes translation average of product over selected pattern refs.
    """
    prod = None

    for j, ref in enumerate(pattern.refs):
        if not (mask & (1 << j)):
            continue

        F = shift_field(X[ref.ori], ref.shift)
        prod = F if prod is None else prod * F

    return float(prod.mean().detach().cpu().item())


# ============================================================
# Moment accumulator with block jackknife
# ============================================================

class MomentAccumulator:
    def __init__(self, patterns: List[Pattern], block_size: int):
        self.patterns = patterns
        self.block_size = int(block_size)

        self.values: Dict[Tuple[str, int], List[float]] = {}
        self.q_values: List[float] = []

        for pat in patterns:
            k = len(pat.refs)
            for mask in nonempty_submasks(k):
                self.values[(pat.name, mask)] = []

    def add_config(self, X: torch.Tensor):
        self.q_values.append(float(X.mean().detach().cpu().item()))

        for pat in self.patterns:
            k = len(pat.refs)
            for mask in nonempty_submasks(k):
                val = pattern_subset_product_mean(X, pat, mask)
                self.values[(pat.name, mask)].append(val)
```

```
    def _moment_mean(self, pat_name: str, mask: int, idxs=None) -> float:
        arr = np.asarray(self.values[(pat_name, mask)], dtype=np.float64)
        if idxs is None:
            return float(arr.mean())
        return float(arr[idxs].mean())

    def _q_mean(self, idxs=None) -> float:
        arr = np.asarray(self.q_values, dtype=np.float64)
        if idxs is None:
            return float(arr.mean())
        return float(arr[idxs].mean())

    def _compute_pattern_stats_for_indices(self, pat: Pattern, idxs=None) -> Dict[str, float]:
        k = len(pat.refs)
        moments = {}
        for mask in nonempty_submasks(k):
            moments[mask] = self._moment_mean(pat.name, mask, idxs)

        kappa = cumulant_from_moments(k, moments)
        q_meas = self._q_mean(idxs)

        return {
            "q_meas_global": q_meas,
            "full_moment": moments[(1 << k) - 1],
            "connected_cumulant": kappa,
            "abs_cumulant": abs(kappa),
            "abs_cumulant_over_qk": abs(kappa) / ((q_meas + EPS) ** k),
            "abs_cumulant_over_q_rooted": abs(kappa) / (q_meas + EPS),
            "signed_cumulant_over_qk": kappa / ((q_meas + EPS) ** k),
            "moments_json": json.dumps({str(mask): moments[mask] for mask in sorted(moments)}),
        }

    def _jackknife_for_pattern(self, pat: Pattern) -> Dict[str, float]:
        n = len(self.q_values)
        if n < 2 * self.block_size:
            return {
                "n_blocks": 0,
                "jk_se_connected_cumulant": np.nan,
                "jk_se_abs_cumulant_over_qk": np.nan,
                "jk_se_abs_cumulant_over_q_rooted": np.nan,
                "jk_mean_connected_cumulant": np.nan,
                "jk_mean_abs_cumulant_over_qk": np.nan,
                "jk_mean_abs_cumulant_over_q_rooted": np.nan,
            }

        n_blocks = n // self.block_size
        usable = n_blocks * self.block_size
        all_idxs = np.arange(usable)

        vals_kappa = []
        vals_qk = []
        vals_rooted = []

        for b in range(n_blocks):
            start = b * self.block_size
            end = (b + 1) * self.block_size
            keep = np.concatenate([all_idxs[:start], all_idxs[end:]])
            stats = self._compute_pattern_stats_for_indices(pat, keep)
            vals_kappa.append(stats["connected_cumulant"])
            vals_qk.append(stats["abs_cumulant_over_qk"])
            vals_rooted.append(stats["abs_cumulant_over_q_rooted"])

        def jk_se(vals):
            vals = np.asarray(vals, dtype=np.float64)
            mean = vals.mean()
            se = math.sqrt((len(vals) - 1) / len(vals) * np.sum((vals - mean) ** 2))
            return float(mean), float(se)

        mean_k, se_k = jk_se(vals_kappa)
        mean_qk, se_qk = jk_se(vals_qk)
        mean_r, se_r = jk_se(vals_rooted)

        return {
            "n_blocks": int(n_blocks),
            "jk_se_connected_cumulant": se_k,
            "jk_se_abs_cumulant_over_qk": se_qk,
```

```
                "jk_se_abs_cumulant_over_q_rooted": se_r,
                "jk_mean_connected_cumulant": mean_k,
                "jk_mean_abs_cumulant_over_qk": mean_qk,
                "jk_mean_abs_cumulant_over_q_rooted": mean_r,
            }

        def finalize_rows(self, meta: Dict[str, Any]) -> List[Dict[str, Any]]:
            rows = []
            n_cfg = len(self.q_values)

            for pat in self.patterns:
                stats = self._compute_pattern_stats_for_indices(pat, None)
                jk = self._jackknife_for_pattern(pat)

                refs_json = json.dumps([
                    {"ori": r.ori, "plane": PLAQUETTE_ORIS[r.ori], "shift": r.shift}
                    for r in pat.refs
                ])

                row = dict(meta)
                row.update({
                    "pattern_name": pat.name,
                    "pattern_kind": pat.kind,
                    "support_size": len(pat.refs),
                    "tau_mst": mst_tau(pat.refs),
                    "refs_json": refs_json,
                    "n_cfg": n_cfg,
                    "block_size": self.block_size,
                })
                row.update(stats)
                row.update(jk)
                rows.append(row)

            return rows


    # ============================================================
    # Run functions
    # ============================================================

    def safe_beta_tag(beta: float) -> str:
        return str(beta).replace(".", "p")


    def field_cache_path(L: int, beta: float, start_mode: str, thermal_sweeps: int) -> str:
        btag = safe_beta_tag(beta)
        return os.path.join(
            FIELD_CACHE_DIR,
            f"v17b_L{L}_beta{btag}_{start_mode}_therm{thermal_sweeps}_seed{SEED}.pt",
        )


    def init_or_load_field(L: int, beta: float, start_mode: str, thermal_sweeps: int):
        cache = field_cache_path(L, beta, start_mode, thermal_sweeps)

        if (not FORCE_RETHERMALIZE) and LOAD_FIELD_CACHE and os.path.exists(cache):
            print(f"[cache] loading thermalized field: {cache}")
            U = torch.load(cache, map_location=DEVICE).to(device=DEVICE, dtype=DTYPE)
            return U, True, cache

        print(f"[init] {start_mode} L={L} beta={beta}")

        if start_mode == "cold":
            U = cold_su2_links(L)
        elif start_mode == "hot":
            U = hot_su2_links(L)
        else:
            raise ValueError(f"Unknown start_mode={start_mode}")

        return U, False, cache


    def initial_sigma(beta: float) -> float:
        if BASE_PROPOSAL_SIGMA is not None:
            return float(BASE_PROPOSAL_SIGMA)
        return float(0.50 / math.sqrt(beta))
```

```
def thermalize(U: torch.Tensor, beta: float, parity_masks, sigma: float, thermal_sweeps: int):
    logs = []
    t0 = time.time()
    adapt_until = int(max(50, ADAPT_UNTIL_FRACTION_OF_THERM * thermal_sweeps))

    for sweep in range(1, thermal_sweeps + 1):
        acc = metropolis_sweep(U, beta, sigma, parity_masks)

        if ADAPT_PROPOSAL and (sweep % ADAPT_EVERY == 0) and sweep <= adapt_until:
            sigma *= math.exp(ADAPT_RATE * (acc - ADAPT_TARGET_ACC))
            sigma = float(np.clip(sigma, 0.02, 1.50))

        if sweep == 1 or sweep % 50 == 0 or sweep == thermal_sweeps:
            print(f"  [therm] sweep={sweep:5d}/{thermal_sweeps} acc={acc:.4f} sigma={sigma:.5f}")

        logs.append({
            "sweep": sweep,
            "acceptance": acc,
            "proposal_sigma": sigma,
            "elapsed_s": time.time() - t0,
        })

    return U, sigma, logs


def collect_calibration_samples(
    U: torch.Tensor,
    beta: float,
    parity_masks,
    sigma: float,
    n_calib_cfg: int,
    between_sweeps: int,
    calib_max_phi_per_cfg: int,
) -> np.ndarray:
    samples = []

    print(f"[calib] collecting {n_calib_cfg} calibration configs")

    for ci in range(1, n_calib_cfg + 1):
        for _ in range(between_sweeps):
            metropolis_sweep(U, beta, sigma, parity_masks)

        phi = plaquette_phi(U)
        flat = phi.flatten()
        n = flat.numel()
        take = min(calib_max_phi_per_cfg, n)
        idx = torch.randint(0, n, (take,), device=DEVICE)
        vals = flat[idx].detach().cpu().numpy()
        samples.append(vals)

        if ci == 1 or ci % 5 == 0 or ci == n_calib_cfg:
            print(f"  [calib] cfg={ci:4d}/{n_calib_cfg}, sampled={take}")

    return np.concatenate(samples, axis=0)


def run_one_combo(L: int, beta: float, start_mode: str):
    settings = L_SETTINGS[L]
    thermal_sweeps = settings["thermal_sweeps"]
    n_calib_cfg = settings["n_calib_cfg"]
    n_meas_cfg = settings["n_meas_cfg"]
    between_sweeps = settings["between_sweeps"]
    calib_max_phi_per_cfg = settings["calib_max_phi_per_cfg"]
    block_size = settings["block_size"]
    r_distances = R_DISTANCES_BY_L.get(L, [1, 2, 3, 4])

    print("=" * 110)
    print(f"[combo] L={L}, beta={beta}, start={start_mode}, mode={RUN_MODE}")
    print("=" * 110)

    parity_masks = [make_parity_mask(L, 0), make_parity_mask(L, 1)]
    sigma = initial_sigma(beta)

    U, loaded, cache = init_or_load_field(L, beta, start_mode, thermal_sweeps)

    thermal_logs = []
```

```
        if not loaded:
            U, sigma, thermal_logs = thermalize(U, beta, parity_masks, sigma, thermal_sweeps)

            if SAVE_FIELD_CACHE:
                torch.save(U.detach().cpu(), cache)
                print(f"[cache] saved thermalized field: {cache}")
        else:
            print("[therm] skipped; using cached field")
            print(f"[sigma] using fresh default proposal sigma={sigma:.5f} after cached load")

        # Calibration phase.
        phi_samples = collect_calibration_samples(
            U=U,
            beta=beta,
            parity_masks=parity_masks,
            sigma=sigma,
            n_calib_cfg=n_calib_cfg,
            between_sweeps=between_sweeps,
            calib_max_phi_per_cfg=calib_max_phi_per_cfg,
        )

        threshold_rows = []
        thresholds = {}

        for eta in ETA_LIST:
            for qtar in Q_TARGET_LIST:
                t = calibrate_threshold_from_samples(phi_samples, eta, qtar)
                thresholds[(eta, qtar)] = t
                achieved = float(sigmoid_np((phi_samples - t) / eta).mean())

                threshold_rows.append({
                    "L": L,
                    "beta": beta,
                    "start_mode": start_mode,
                    "eta": eta,
                    "q_target": qtar,
                    "threshold_t": t,
                    "q_achieved_on_calib_samples": achieved,
                    "n_phi_calib_samples": len(phi_samples),
                })

                print(
                    f"  [threshold] eta={eta:.5g} qtar={qtar:.5g} "
                    f"t={t:.8f} q_calib={achieved:.8g}"
                )

        # Patterns and accumulators.
        patterns = build_patterns(L, r_distances)
        print(f"[patterns] {len(patterns)} support patterns")

        accumulators = {
            (eta, qtar): MomentAccumulator(patterns, block_size=block_size)
            for eta in ETA_LIST
            for qtar in Q_TARGET_LIST
        }

        meas_logs = []
        t0 = time.time()

        print(f"[meas] collecting {n_meas_cfg} measurement configs")

        for mi in range(1, n_meas_cfg + 1):
            accs = []
            for _ in range(between_sweeps):
                accs.append(metropolis_sweep(U, beta, sigma, parity_masks))

            phi = plaquette_phi(U)

            for eta in ETA_LIST:
                for qtar in Q_TARGET_LIST:
                    t = thresholds[(eta, qtar)]
                    X = smooth_X_from_phi(phi, t, eta)
                    accumulators[(eta, qtar)].add_config(X)

            mean_acc = float(np.mean(accs))

            if mi == 1 or mi % 20 == 0 or mi == n_meas_cfg:
```

```
                print(
                    f"  [meas] cfg={mi:5d}/{n_meas_cfg} "
                    f"acc={mean_acc:.4f} elapsed={time.time()-t0:.1f}s"
                )

            meas_logs.append({
                "L": L,
                "beta": beta,
                "start_mode": start_mode,
                "meas_cfg": mi,
                "acceptance_mean_between": mean_acc,
                "proposal_sigma": sigma,
                "elapsed_s": time.time() - t0,
            })

        if SAVE_FIELD_CACHE:
            torch.save(U.detach().cpu(), cache)
            print(f"[cache] updated field: {cache}")

        cumulant_rows = []

        for eta in ETA_LIST:
            for qtar in Q_TARGET_LIST:
                meta = {
                    "run_id": RUN_ID,
                    "run_mode": RUN_MODE,
                    "L": L,
                    "beta": beta,
                    "start_mode": start_mode,
                    "eta": eta,
                    "q_target": qtar,
                    "threshold_t": thresholds[(eta, qtar)],
                    "thermal_sweeps": thermal_sweeps,
                    "between_sweeps": between_sweeps,
                    "n_calib_cfg": n_calib_cfg,
                    "n_meas_cfg": n_meas_cfg,
                }

                cumulant_rows.extend(accumulators[(eta, qtar)].finalize_rows(meta))

        return threshold_rows, cumulant_rows, thermal_logs, meas_logs


    # ============================================================
    # Output / reporting
    # ============================================================

    def make_summary(cumulants_df: pd.DataFrame) -> pd.DataFrame:
        if cumulants_df.empty:
            return pd.DataFrame()

        group_cols = [
            "L", "beta", "start_mode", "eta", "q_target", "support_size", "pattern_kind"
        ]

        rows = []

        for keys, g in cumulants_df.groupby(group_cols):
            d = dict(zip(group_cols, keys))
            d.update({
                "n_patterns": int(len(g)),
                "max_abs_cumulant_over_qk": float(g["abs_cumulant_over_qk"].max()),
                "mean_abs_cumulant_over_qk": float(g["abs_cumulant_over_qk"].mean()),
                "max_abs_cumulant_over_q_rooted": float(g["abs_cumulant_over_q_rooted"].max()),
                "mean_abs_cumulant_over_q_rooted": float(g["abs_cumulant_over_q_rooted"].mean()),
                "max_abs_cumulant": float(g["abs_cumulant"].max()),
                "mean_abs_cumulant": float(g["abs_cumulant"].mean()),
                "mean_q_meas_global": float(g["q_meas_global"].mean()),
                "max_jk_se_abs_cumulant_over_qk": float(g["jk_se_abs_cumulant_over_qk"].max(skipna=True)),
                "max_jk_se_abs_cumulant_over_q_rooted": float(g["jk_se_abs_cumulant_over_q_rooted"].max(skipna=True)),
            })
            rows.append(d)

        return pd.DataFrame(rows)


    def make_pair_decay(cumulants_df: pd.DataFrame) -> pd.DataFrame:
```

```
    rows = []

    if cumulants_df.empty:
        return pd.DataFrame()

    pair_df = cumulants_df[cumulants_df["support_size"] == 2].copy()
    if pair_df.empty:
        return pd.DataFrame()

    group_cols = ["L", "beta", "start_mode", "eta", "q_target", "pattern_kind"]

    for keys, g in pair_df.groupby(group_cols):
        d = dict(zip(group_cols, keys))

        # Axis-distance patterns only.
        if "axis" not in d["pattern_kind"]:
            continue

        gg = g[g["pattern_name"].str.contains("_r", regex=False)].copy()
        if len(gg) < 2:
            continue

        rs = []
        ys = []

        for _, row in gg.iterrows():
            name = row["pattern_name"]
            try:
                r = int(name.split("_r")[-1])
            except Exception:
                continue

            y = float(row["abs_cumulant_over_qk"])
            if y > 0 and np.isfinite(y):
                rs.append(r)
                ys.append(y)

        if len(rs) >= 2:
            x = np.asarray(rs, dtype=np.float64)
            ly = np.log(np.asarray(ys, dtype=np.float64))
            slope, intercept = np.polyfit(x, ly, 1)

            d.update({
                "n_points": int(len(rs)),
                "r_values_json": json.dumps([int(v) for v in rs]),
                "y_values_json": json.dumps([float(v) for v in ys]),
                "log_decay_slope_vs_r": float(slope),
                "log_decay_intercept": float(intercept),
                "decay_rate_positive": float(-slope),
                "is_decay_negative_slope": bool(slope < 0),
            })
            rows.append(d)

    return pd.DataFrame(rows)


def make_block_jackknife_diagnostics(cumulants_df: pd.DataFrame) -> pd.DataFrame:
    if cumulants_df.empty:
        return pd.DataFrame()

    rows = []
    for _, row in cumulants_df.iterrows():
        se = row.get("jk_se_abs_cumulant_over_qk", np.nan)
        val = row.get("abs_cumulant_over_qk", np.nan)

        if pd.isna(se) or pd.isna(val) or val <= 0:
            rel = np.nan
        else:
            rel = float(se / (abs(val) + EPS))

        rows.append({
            "L": row["L"],
            "beta": row["beta"],
            "start_mode": row["start_mode"],
            "eta": row["eta"],
            "q_target": row["q_target"],
            "pattern_name": row["pattern_name"],
```

```
                "pattern_kind": row["pattern_kind"],
                "support_size": row["support_size"],
                "tau_mst": row["tau_mst"],
                "n_blocks": row["n_blocks"],
                "abs_cumulant_over_qk": val,
                "jk_se_abs_cumulant_over_qk": se,
                "rel_jk_se_abs_cumulant_over_qk": rel,
                "abs_cumulant_over_q_rooted": row["abs_cumulant_over_q_rooted"],
                "jk_se_abs_cumulant_over_q_rooted": row["jk_se_abs_cumulant_over_q_rooted"],
            })

    return pd.DataFrame(rows)


def write_readout(
    readout_md: str,
    cumulants_df: pd.DataFrame,
    summary_df: pd.DataFrame,
    pair_decay_df: pd.DataFrame,
    jk_df: pd.DataFrame,
    paths: Dict[str, str],
):
    lines = []
    lines.append(f"# {RUN_ID}")
    lines.append("")
    lines.append("## Purpose")
    lines.append("")
    lines.append("Production numerical diagnostic for BS = Balaban smooth-source expansion.")
    lines.append("")
    lines.append("The measured quantity is the connected square-free cumulant of smoothed plaquette defects.")
    lines.append("")
    lines.append("This run does not prove BS. It tests whether the expected connected coefficient hierarchy is numerically plau
    lines.append("")
    lines.append("## Interpretation target")
    lines.append("")
    lines.append("Positive BS shadow:")
    lines.append("")
    lines.append("1. Pair coefficients decay with distance.")
    lines.append("2. Far controls are small relative to local incident supports.")
    lines.append("3. Normalized cumulants do not grow badly with L.")
    lines.append("4. Smaller eta does not cause uncontrolled blow-up.")
    lines.append("5. Rooted burden remains moderate.")
    lines.append("")
    lines.append("## Files")
    lines.append("")
    for label, path in paths.items():
        lines.append(f"- `{os.path.basename(path)}`")
    lines.append("")

    if not summary_df.empty:
        lines.append("## Worst aggregate rows by max abs_cumulant_over_qk")
        lines.append("")
        worst = summary_df.sort_values("max_abs_cumulant_over_qk", ascending=False).head(30)
        lines.append(worst.to_markdown(index=False))
        lines.append("")

    if not pair_decay_df.empty:
        lines.append("## Pair decay fits")
        lines.append("")
        lines.append(pair_decay_df.to_markdown(index=False))
        lines.append("")

    if not jk_df.empty:
        lines.append("## Worst relative jackknife uncertainties")
        lines.append("")
        tmp = jk_df.copy()
        tmp = tmp[np.isfinite(tmp["rel_jk_se_abs_cumulant_over_qk"])]
        if not tmp.empty:
            worst_jk = tmp.sort_values("rel_jk_se_abs_cumulant_over_qk", ascending=False).head(30)
            lines.append(worst_jk.to_markdown(index=False))
            lines.append("")

    if not cumulants_df.empty:
        lines.append("## Worst individual cumulant rows")
        lines.append("")
        cols = [
            "L", "beta", "start_mode", "eta", "q_target", "pattern_name",
```

```
                "support_size", "tau_mst", "q_meas_global",
                "abs_cumulant_over_qk", "abs_cumulant_over_q_rooted",
                "jk_se_abs_cumulant_over_qk",
            ]
            worst_ind = cumulants_df.sort_values("abs_cumulant_over_qk", ascending=False).head(30)
            lines.append(worst_ind[cols].to_markdown(index=False))
            lines.append("")

        with open(readout_md, "w") as f:
            f.write("\n".join(lines))


    def write_outputs(all_thresholds, all_cumulants, all_thermal_logs, all_meas_logs):
        thresholds_df = pd.DataFrame(all_thresholds)
        cumulants_df = pd.DataFrame(all_cumulants)
        thermal_df = pd.DataFrame(all_thermal_logs)
        meas_df = pd.DataFrame(all_meas_logs)

        summary_df = make_summary(cumulants_df)
        pair_decay_df = make_pair_decay(cumulants_df)
        jk_df = make_block_jackknife_diagnostics(cumulants_df)

        thresholds_csv = os.path.join(OUTDIR, "thresholds.csv")
        cumulants_csv = os.path.join(OUTDIR, "connected_cumulants.csv")
        thermal_csv = os.path.join(OUTDIR, "thermalization_logs.csv")
        meas_csv = os.path.join(OUTDIR, "measurement_logs.csv")
        summary_csv = os.path.join(OUTDIR, "summary_by_pattern_kind.csv")
        pair_decay_csv = os.path.join(OUTDIR, "pair_decay_fits.csv")
        jk_csv = os.path.join(OUTDIR, "block_jackknife_diagnostics.csv")
        metadata_json = os.path.join(OUTDIR, "metadata.json")
        readout_md = os.path.join(OUTDIR, "READOUT.md")

        thresholds_df.to_csv(thresholds_csv, index=False)
        cumulants_df.to_csv(cumulants_csv, index=False)
        thermal_df.to_csv(thermal_csv, index=False)
        meas_df.to_csv(meas_csv, index=False)
        summary_df.to_csv(summary_csv, index=False)
        pair_decay_df.to_csv(pair_decay_csv, index=False)
        jk_df.to_csv(jk_csv, index=False)

        metadata = {
            "run_id": RUN_ID,
            "run_tag": RUN_TAG,
            "run_mode": RUN_MODE,
            "seed": SEED,
            "device": str(DEVICE),
            "cuda_device_name": torch.cuda.get_device_name(0) if torch.cuda.is_available() else None,
            "L_LIST": L_LIST,
            "BETA_LIST": BETA_LIST,
            "START_MODES": START_MODES,
            "ETA_LIST": ETA_LIST,
            "Q_TARGET_LIST": Q_TARGET_LIST,
            "R_DISTANCES_BY_L": R_DISTANCES_BY_L,
            "L_SETTINGS": L_SETTINGS,
            "BASE_PROPOSAL_SIGMA": BASE_PROPOSAL_SIGMA,
            "ADAPT_PROPOSAL": ADAPT_PROPOSAL,
            "ADAPT_TARGET_ACC": ADAPT_TARGET_ACC,
            "ADAPT_RATE": ADAPT_RATE,
            "FORCE_RETHERMALIZE": FORCE_RETHERMALIZE,
            "SAVE_FIELD_CACHE": SAVE_FIELD_CACHE,
            "LOAD_FIELD_CACHE": LOAD_FIELD_CACHE,
            "outdir": OUTDIR,
            "field_cache_dir": FIELD_CACHE_DIR,
            "purpose": "Finite-volume numerical diagnostic for Balaban smooth-source expansion via connected cumulants.",
            "does_not_prove": [
                "analytic BS cluster expansion",
                "uniform-in-L convergence",
                "uniform-in-eta hard-threshold bridge",
                "top-p de-Poissonization",
                "Yang-Mills mass gap",
            ],
        }

        with open(metadata_json, "w") as f:
            json.dump(metadata, f, indent=2)

        path_map = {
```

```
            "thresholds": thresholds_csv,
            "connected_cumulants": cumulants_csv,
            "summary": summary_csv,
            "pair_decay": pair_decay_csv,
            "jackknife": jk_csv,
            "thermalization": thermal_csv,
            "measurement": meas_csv,
            "metadata": metadata_json,
        }

        write_readout(
            readout_md=readout_md,
            cumulants_df=cumulants_df,
            summary_df=summary_df,
            pair_decay_df=pair_decay_df,
            jk_df=jk_df,
            paths=path_map,
        )

        zip_path = os.path.join(BASE_OUTDIR, f"{RUN_ID}.zip")
        with zipfile.ZipFile(zip_path, "w", compression=zipfile.ZIP_DEFLATED) as zf:
            for root, _, files in os.walk(OUTDIR):
                for fn in files:
                    path = os.path.join(root, fn)
                    arc = os.path.relpath(path, OUTDIR)
                    zf.write(path, arcname=arc)

        print("=" * 110)
        print("[outputs written]")
        print(f"OUTDIR: {OUTDIR}")
        print(f"ZIP:    {zip_path}")
        print("=" * 110)

        return {
            "thresholds_csv": thresholds_csv,
            "cumulants_csv": cumulants_csv,
            "thermal_csv": thermal_csv,
            "meas_csv": meas_csv,
            "summary_csv": summary_csv,
            "pair_decay_csv": pair_decay_csv,
            "jk_csv": jk_csv,
            "metadata_json": metadata_json,
            "readout_md": readout_md,
            "zip_path": zip_path,
        }


    # ============================================================
    # Main
    # ============================================================

    def main():
        all_thresholds = []
        all_cumulants = []
        all_thermal_logs = []
        all_meas_logs = []

        t_all = time.time()

        print("=" * 110)
        print("[configuration]")
        print(f"RUN_MODE={RUN_MODE}")
        print(f"L_LIST={L_LIST}")
        print(f"BETA_LIST={BETA_LIST}")
        print(f"START_MODES={START_MODES}")
        print(f"ETA_LIST={ETA_LIST}")
        print(f"Q_TARGET_LIST={Q_TARGET_LIST}")
        print(f"FORCE_RETHERMALIZE={FORCE_RETHERMALIZE}")
        print("=" * 110)

        for L in L_LIST:
            for beta in BETA_LIST:
                for start_mode in START_MODES:
                    threshold_rows, cumulant_rows, thermal_logs, meas_logs = run_one_combo(L, beta, start_mode)

                    for r in threshold_rows:
                        r["run_id"] = RUN_ID
```

```
                        r["run_mode"] = RUN_MODE

                    for r in thermal_logs:
                        r["run_id"] = RUN_ID
                        r["run_mode"] = RUN_MODE
                        r["L"] = L
                        r["beta"] = beta
                        r["start_mode"] = start_mode

                    for r in meas_logs:
                        r["run_id"] = RUN_ID
                        r["run_mode"] = RUN_MODE

                    all_thresholds.extend(threshold_rows)
                    all_cumulants.extend(cumulant_rows)
                    all_thermal_logs.extend(thermal_logs)
                    all_meas_logs.extend(meas_logs)

                    # Incremental output after each combo.
                    write_outputs(all_thresholds, all_cumulants, all_thermal_logs, all_meas_logs)

        paths = write_outputs(all_thresholds, all_cumulants, all_thermal_logs, all_meas_logs)

        print(f"[done] elapsed_total_s={time.time() - t_all:.2f}")
        print("[important]")
        print("Upload the ZIP and READOUT.md for review.")
        print(paths["zip_path"])


    if __name__ == "__main__":
        main()
```

```
  [therm] sweep=  700/800 acc=0.4946 sigma=0.21344
```

```
  [therm] sweep=  700/800 acc=0.4946 sigma=0.21344
  [therm] sweep=  750/800 acc=0.4948 sigma=0.21344
  [therm] sweep=  800/800 acc=0.4944 sigma=0.21344
[cache] saved thermalized field: /content/PMBSF_v17b_thermalized_fields/v17b_L24_beta4p0_hot_therm800_seed23061721.pt
[calib] collecting 20 calibration configs
  [calib] cfg=   1/20, sampled=300000
  [calib] cfg=   5/20, sampled=300000
  [calib] cfg=  10/20, sampled=300000
  [calib] cfg=  15/20, sampled=300000
```

```
# ============================================================
# L=64 Critical Threshold Law — PRINT ONLY
# No file saving. No Google Drive. No plots.
# ============================================================

import numpy as np
import pandas as pd

from sklearn.model_selection import GroupShuffleSplit, LeaveOneGroupOut
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LinearRegression

# ----------------------------
# 0. Config
# ----------------------------

SEED = 6402
rng = np.random.default_rng(SEED)

L = 64
N = L * L
N_DIMERS = 64
M = 2 * N_DIMERS
K_LOW = 128
N_MASKS_PER_FAMILY = 30

FAMILIES = [
    "random_dimers",
    "stripe_dimers",
    "ring_dimers",
    "lowmode_biased_x",
    "lowmode_biased_diag",
    "blue_noise_dimers",
]

# ----------------------------
# 1. Fourier low modes
# ----------------------------

def lap_eigenvalue(kx, ky, L):
    return 4.0 - 2.0 * np.cos(2 * np.pi * kx / L) - 2.0 * np.cos(2 * np.pi * ky / L)

modes = []
for kx in range(L):
    for ky in range(L):
        if kx == 0 and ky == 0:
            continue
        modes.append((lap_eigenvalue(kx, ky, L), kx, ky))

modes = sorted(modes, key=lambda x: x[0])
selected = modes[:K_LOW]

evals = np.array([x[0] for x in selected], dtype=float)
kxs = np.array([x[1] for x in selected], dtype=int)
kys = np.array([x[2] for x in selected], dtype=int)

lambda1 = float(evals[0])
lambda_max = float(evals[-1])

xs = np.repeat(np.arange(L), L)
ys = np.tile(np.arange(L), L)
phase = 2 * np.pi * (xs[:, None] * kxs[None, :] + ys[:, None] * kys[None, :]) / L
Phi = np.exp(1j * phase) / np.sqrt(N)

# ----------------------------
```

```
# 2. Periodic geometry
# ----------------------------

def torus_dist2(a, b):
    ax, ay = a
    bx, by = b
    dx = abs(ax - bx)
    dy = abs(ay - by)
    dx = min(dx, L - dx)
    dy = min(dy, L - dy)
    return dx * dx + dy * dy

def dimer_sites(center, orient):
    x, y = center
    if orient == 0:
        return [(x, y), ((x + 1) % L, y)]
    return [(x, y), (x, (y + 1) % L)]

def mask_from_dimers(dimers):
    arr = np.zeros((L, L), dtype=bool)
    for center, orient in dimers:
        for x, y in dimer_sites(center, orient):
            arr[x, y] = True
    return arr.ravel()

def periodic_component_sizes(mask):
    arr = mask.reshape(L, L)
    seen = np.zeros((L, L), dtype=bool)
    sizes = []

    for x0 in range(L):
        for y0 in range(L):
            if not arr[x0, y0] or seen[x0, y0]:
                continue

            stack = [(x0, y0)]
            seen[x0, y0] = True
            size = 0

            while stack:
                x, y = stack.pop()
                size += 1
                for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
                    xx = (x + dx) % L
                    yy = (y + dy) % L
                    if arr[xx, yy] and not seen[xx, yy]:
                        seen[xx, yy] = True
                        stack.append((xx, yy))

            sizes.append(size)

    return sizes

def cluster_count(mask):
    return len(periodic_component_sizes(mask))

def largest_cluster(mask):
    sizes = periodic_component_sizes(mask)
    return max(sizes) if sizes else 0

def valid_fixed_local_geometry(mask):
    return (
        int(mask.sum()) == M
        and cluster_count(mask) == N_DIMERS
        and largest_cluster(mask) == 2
    )

def mean_pair_distance(mask):
    pts = np.argwhere(mask.reshape(L, L))
    vals = []
    for i in range(len(pts)):
        for j in range(i + 1, len(pts)):
            vals.append(np.sqrt(torus_dist2(tuple(pts[i]), tuple(pts[j]))))
    return float(np.mean(vals)) if vals else 0.0

def low_fourier_amplitude(mask, kx, ky):
    arr = mask.reshape(L, L).astype(float)
```

```
        X = np.arange(L)[:, None]
        Y = np.arange(L)[None, :]
        ph = 2 * np.pi * (kx * X + ky * Y) / L
        coeff = np.sum(arr * np.exp(-1j * ph)) / max(mask.sum(), 1)
        return float(abs(coeff))

    # ----------------------------
    # 3. Mask generators
    # ----------------------------

    def try_build_dimers(candidate_centers, orientation_rule, min_center_dist=3):
        dimers = []
        used = np.zeros((L, L), dtype=bool)

        for center in candidate_centers:
            orient = orientation_rule(center)
            sites = dimer_sites(center, orient)

            if any(used[x, y] for x, y in sites):
                continue

            ok = True
            for c_prev, _ in dimers:
                if torus_dist2(center, c_prev) < min_center_dist ** 2:
                    ok = False
                    break
            if not ok:
                continue

            for x, y in sites:
                used[x, y] = True

            dimers.append((center, orient))
            if len(dimers) == N_DIMERS:
                break

        if len(dimers) < N_DIMERS:
            return None

        mask = mask_from_dimers(dimers)
        if not valid_fixed_local_geometry(mask):
            return None

        return mask

    def generate_random_dimers():
        for _ in range(5000):
            centers = [(rng.integers(0, L), rng.integers(0, L)) for _ in range(3000)]
            rng.shuffle(centers)
            mask = try_build_dimers(
                centers,
                orientation_rule=lambda c: int(rng.integers(0, 2)),
                min_center_dist=3,
            )
            if mask is not None:
                return mask
        raise RuntimeError("failed random_dimers")

    def generate_stripe_dimers():
        for _ in range(5000):
            y0 = rng.integers(0, L)
            band_width = 10
            centers = []
            for _j in range(5000):
                y = int((y0 + rng.integers(-band_width//2, band_width//2 + 1)) % L)
                x = int(rng.integers(0, L))
                centers.append((x, y))
            rng.shuffle(centers)

            mask = try_build_dimers(
                centers,
                orientation_rule=lambda c: 0,
                min_center_dist=3,
            )
            if mask is not None:
                return mask
        raise RuntimeError("failed stripe_dimers")
```

```
    def generate_ring_dimers():
        for _ in range(500):
            cx, cy = rng.uniform(0, L), rng.uniform(0, L)
            radius = rng.uniform(16, 24)
            centers = []

            for _j in range(6000):
                theta = rng.uniform(0, 2*np.pi)
                rr = radius + rng.normal(0, 2.0)
                x = int(np.round(cx + rr * np.cos(theta))) % L
                y = int(np.round(cy + rr * np.sin(theta))) % L
                centers.append((x, y))

            rng.shuffle(centers)
            mask = try_build_dimers(
                centers,
                orientation_rule=lambda c: int(rng.integers(0, 2)),
                min_center_dist=3,
            )
            if mask is not None:
                return mask

        return generate_random_dimers()

    def generate_lowmode_biased_x():
        centers = []
        for _ in range(12000):
            x = rng.integers(0, L)
            y = rng.integers(0, L)
            weight = 1.0 + 0.90 * np.cos(2*np.pi*x/L)
            if rng.uniform(0, 1.90) <= weight:
                centers.append((x, y))

        rng.shuffle(centers)
        mask = try_build_dimers(
            centers,
            orientation_rule=lambda c: int(rng.integers(0, 2)),
            min_center_dist=3,
        )
        if mask is not None:
            return mask
        return generate_random_dimers()

    def generate_lowmode_biased_diag():
        centers = []
        for _ in range(12000):
            x = rng.integers(0, L)
            y = rng.integers(0, L)
            weight = 1.0 + 0.90 * np.cos(2*np.pi*(x + y)/L)
            if rng.uniform(0, 1.90) <= weight:
                centers.append((x, y))

        rng.shuffle(centers)
        mask = try_build_dimers(
            centers,
            orientation_rule=lambda c: int(rng.integers(0, 2)),
            min_center_dist=3,
        )
        if mask is not None:
            return mask
        return generate_random_dimers()

    def generate_blue_noise_dimers():
        centers = []
        all_sites = [(x, y) for x in range(L) for y in range(L)]
        centers.append((rng.integers(0, L), rng.integers(0, L)))

        while len(centers) < N_DIMERS:
            candidate_idxs = rng.choice(len(all_sites), size=700, replace=False)
            best = None
            best_score = -1
            for idx in candidate_idxs:
                c = all_sites[idx]
                score = min(torus_dist2(c, q) for q in centers)
                if score > best_score:
                    best_score = score
```

```
                    best = c
            centers.append(best)

        candidates = centers + [(rng.integers(0, L), rng.integers(0, L)) for _ in range(3000)]
        mask = try_build_dimers(
            candidates,
            orientation_rule=lambda c: int(rng.integers(0, 2)),
            min_center_dist=3,
        )
        if mask is not None:
            return mask
        return generate_random_dimers()

    def generate_mask_family(family):
        if family == "random_dimers":
            return generate_random_dimers()
        if family == "stripe_dimers":
            return generate_stripe_dimers()
        if family == "ring_dimers":
            return generate_ring_dimers()
        if family == "lowmode_biased_x":
            return generate_lowmode_biased_x()
        if family == "lowmode_biased_diag":
            return generate_lowmode_biased_diag()
        if family == "blue_noise_dimers":
            return generate_blue_noise_dimers()
        raise ValueError(family)

    # ----------------------------
    # 4. Projected operators
    # ----------------------------

    def hermitian(A):
        return 0.5 * (A + A.conj().T)

    def eigmax_hermitian(A):
        return float(np.linalg.eigvalsh(hermitian(A)).max().real)

    def eigmin_hermitian(A):
        return float(np.linalg.eigvalsh(hermitian(A)).min().real)

    def projected_matrix(mask):
        W = Phi[mask, :]
        return hermitian(W.conj().T @ W)

    def R_capacity(G):
        return eigmax_hermitian(G)

    def B0_score(G):
        invsqrt = 1.0 / np.sqrt(evals)
        B = (invsqrt[:, None] * G) * invsqrt[None, :]
        return eigmax_hermitian(B)

    def Hlow_min_eig(G, V):
        H = np.diag(evals).astype(complex) - V * G
        return eigmin_hermitian(H)

    def critical_check(G, Vc):
        return Hlow_min_eig(G, Vc)

    def low_mode_alignment_features(G):
        vals, vecs = np.linalg.eigh(hermitian(G))
        top = vecs[:, np.argmax(vals)]
        weights = np.abs(top)**2
        weights = weights / weights.sum()

        e_mean = float(np.sum(weights * evals))
        e_harm = float(1.0 / np.sum(weights / evals))
        e_lowband = float(weights[evals <= 4 * lambda1].sum())
        return e_mean, e_harm, e_lowband

    # ----------------------------
    # 5. Generate masks and compute thresholds
    # ----------------------------

    rows = []
    mask_id = 0
```

```
total_masks = len(FAMILIES) * N_MASKS_PER_FAMILY

for family in FAMILIES:
    for rep in range(N_MASKS_PER_FAMILY):
        mask = generate_mask_family(family)

        if not valid_fixed_local_geometry(mask):
            raise RuntimeError(f"invalid mask in {family}")

        G = projected_matrix(mask)
        R = R_capacity(G)
        B0 = B0_score(G)

        Vc_BS = 1.0 / B0
        Vc_R = lambda1 / R
        threshold_residual = critical_check(G, Vc_BS)
        e_mean, e_harm, e_lowband = low_mode_alignment_features(G)

        rows.append({
            "mask_id": mask_id,
            "family": family,
            "rep": rep,
            "L": L,
            "N": N,
            "K_LOW": K_LOW,
            "lambda1": lambda1,
            "lambda_max": lambda_max,
            "m": int(mask.sum()),
            "density": float(mask.mean()),
            "cluster_count": cluster_count(mask),
            "largest_cluster": largest_cluster(mask),
            "mean_pair_distance": mean_pair_distance(mask),
            "fourier_10": low_fourier_amplitude(mask, 1, 0),
            "fourier_01": low_fourier_amplitude(mask, 0, 1),
            "fourier_11": low_fourier_amplitude(mask, 1, 1),
            "fourier_20": low_fourier_amplitude(mask, 2, 0),
            "fourier_02": low_fourier_amplitude(mask, 0, 2),
            "R_nonzero": R,
            "B0": B0,
            "Vc_BS_exact": Vc_BS,
            "Vc_R_scalar": Vc_R,
            "ratio_VcR_over_VcBS": Vc_R / Vc_BS,
            "log_Vc_BS": np.log(Vc_BS),
            "log_Vc_R": np.log(Vc_R),
            "critical_residual_min_eig": threshold_residual,
            "topG_energy_mean": e_mean,
            "topG_energy_harmonic": e_harm,
            "topG_low_band_mass": e_lowband,
        })

        mask_id += 1
        if mask_id % 15 == 0:
            print(f"[progress] completed masks {mask_id}/{total_masks}")

df = pd.DataFrame(rows)

# ----------------------------
# 6. Scalar calibration
# ----------------------------

cal = LinearRegression()
cal.fit(df[["log_Vc_R"]].values, df["log_Vc_BS"].values)
df["log_Vc_R_calibrated"] = cal.predict(df[["log_Vc_R"]].values)
df["Vc_R_calibrated"] = np.exp(df["log_Vc_R_calibrated"])

# ----------------------------
# 7. CV setup
# ----------------------------

local_features = ["m", "density", "cluster_count", "largest_cluster"]

geometry_features = local_features + [
    "mean_pair_distance",
    "fourier_10",
    "fourier_01",
    "fourier_11",
    "fourier_20",
```

```
    "fourier_02",
]

scalar_features = geometry_features + [
    "R_nonzero",
    "Vc_R_scalar",
    "log_Vc_R",
]

alignment_features = scalar_features + [
    "topG_energy_mean",
    "topG_energy_harmonic",
    "topG_low_band_mass",
]

feature_sets = [
    ("local_only", local_features),
    ("geometry_no_capacity", geometry_features),
    ("scalar_capacity", scalar_features),
    ("scalar_plus_alignment", alignment_features),
]

def mask_heldout_cv(target="log_Vc_BS", n_splits=8):
    out = []
    groups = df["mask_id"].values
    y = df[target].values

    splitter = GroupShuffleSplit(
        n_splits=n_splits,
        test_size=0.30,
        random_state=SEED,
    )

    for label, features in feature_sets:
        X = df[features].values
        for split_id, (tr, te) in enumerate(splitter.split(X, y, groups)):
            model = RandomForestRegressor(
                n_estimators=500,
                min_samples_leaf=3,
                random_state=SEED + split_id,
                n_jobs=-1,
            )
            model.fit(X[tr], y[tr])
            pred = model.predict(X[te])
            out.append({
                "target": target,
                "model": label,
                "split": split_id,
                "MAE": mean_absolute_error(y[te], pred),
                "R2": r2_score(y[te], pred),
            })

    return pd.DataFrame(out)

def family_heldout_cv(target="log_Vc_BS"):
    out = []
    groups = df["family"].values
    y = df[target].values
    logo = LeaveOneGroupOut()

    for label, features in feature_sets:
        X = df[features].values
        for split_id, (tr, te) in enumerate(logo.split(X, y, groups)):
            fam = df.iloc[te]["family"].iloc[0]
            model = RandomForestRegressor(
                n_estimators=500,
                min_samples_leaf=3,
                random_state=SEED + split_id,
                n_jobs=-1,
            )
            model.fit(X[tr], y[tr])
            pred = model.predict(X[te])
            out.append({
                "target": target,
                "model": label,
                "heldout_family": fam,
                "MAE": mean_absolute_error(y[te], pred),
```

```
                "R2": r2_score(y[te], pred),
            })

    return pd.DataFrame(out)

mask_cv_log = mask_heldout_cv("log_Vc_BS")
family_cv_log = family_heldout_cv("log_Vc_BS")

# ----------------------------
# 8. Ridge law
# ----------------------------

ridge_features = [
    "mean_pair_distance",
    "fourier_10",
    "fourier_01",
    "fourier_11",
    "fourier_20",
    "fourier_02",
    "R_nonzero",
    "log_Vc_R",
    "topG_energy_mean",
    "topG_energy_harmonic",
    "topG_low_band_mass",
]

X_ridge = df[ridge_features].values
y_ridge = df["log_Vc_BS"].values

ridge = Pipeline([
    ("scale", StandardScaler()),
    ("ridge", RidgeCV(alphas=np.logspace(-5, 5, 51))),
])

ridge.fit(X_ridge, y_ridge)
pred_ridge = ridge.predict(X_ridge)

coef = pd.DataFrame({
    "feature": ridge_features,
    "coef_scaled": ridge.named_steps["ridge"].coef_,
}).sort_values("coef_scaled", key=np.abs, ascending=False)

# ============================================================
# 9. PRINT-ONLY READOUT
# ============================================================

print("\n" + "="*100)
print("1. RUN HEADER")
print("="*100)
print("L:", L)
print("N:", N)
print("K_LOW:", K_LOW)
print("lambda1:", lambda1)
print("lambda_max:", lambda_max)

print("\n" + "="*100)
print("2. FIXED GEOMETRY SANITY")
print("="*100)
print(df[["m", "density", "cluster_count", "largest_cluster"]].drop_duplicates().to_string(index=False))

print("\n" + "="*100)
print("3. FAMILY THRESHOLD SUMMARY")
print("="*100)
print(df.groupby("family").agg(
    n=("mask_id", "size"),
    R_mean=("R_nonzero", "mean"),
    R_std=("R_nonzero", "std"),
    B0_mean=("B0", "mean"),
    B0_std=("B0", "std"),
    Vc_BS_mean=("Vc_BS_exact", "mean"),
    Vc_BS_std=("Vc_BS_exact", "std"),
    Vc_R_mean=("Vc_R_scalar", "mean"),
    Vc_R_std=("Vc_R_scalar", "std"),
    ratio_mean=("ratio_VcR_over_VcBS", "mean"),
    ratio_std=("ratio_VcR_over_VcBS", "std"),
).to_string())
```

```
print("\n" + "="*100)
print("4. GLOBAL THRESHOLD DIAGNOSTICS")
print("="*100)
print("corr(Vc_R, Vc_BS):", df["Vc_R_scalar"].corr(df["Vc_BS_exact"]))
print("corr(log Vc_R, log Vc_BS):", df["log_Vc_R"].corr(df["log_Vc_BS"]))
print("MAE Vc_R vs Vc_BS:", mean_absolute_error(df["Vc_BS_exact"], df["Vc_R_scalar"]))
print("MAE log Vc_R vs log Vc_BS:", mean_absolute_error(df["log_Vc_BS"], df["log_Vc_R"]))
print("\nratio Vc_R / Vc_BS:")
print(df["ratio_VcR_over_VcBS"].describe().to_string())

print("\n" + "="*100)
print("5. CALIBRATED SCALAR LAW")
print("="*100)
print("log Vc_BS = %.8f + %.8f log Vc_R" % (cal.intercept_, cal.coef_[0]))
print("R2 log:", r2_score(df["log_Vc_BS"], df["log_Vc_R_calibrated"]))
print("MAE log:", mean_absolute_error(df["log_Vc_BS"], df["log_Vc_R_calibrated"]))
print("MAE raw Vc:", mean_absolute_error(df["Vc_BS_exact"], df["Vc_R_calibrated"]))

print("\n" + "="*100)
print("6. MASK-HELDOUT CV SUMMARY — target log_Vc_BS")
print("="*100)
print(mask_cv_log.groupby("model").agg(
    MAE=("MAE", "mean"),
    R2=("R2", "mean"),
).reset_index().sort_values("MAE").to_string(index=False))

print("\n" + "="*100)
print("7. FAMILY-HELDOUT CV SUMMARY — target log_Vc_BS")
print("="*100)
print(family_cv_log.groupby("model").agg(
    MAE=("MAE", "mean"),
    R2=("R2", "mean"),
).reset_index().sort_values("MAE").to_string(index=False))

print("\n" + "="*100)
print("8. FAMILY-HELDOUT DETAILS — target log_Vc_BS")
print("="*100)
print(family_cv_log.sort_values(
    ["heldout_family", "MAE"]
).to_string(index=False))

print("\n" + "="*100)
print("9. RIDGE LAW FOR log Vc_BS")
print("="*100)
print("alpha:", ridge.named_steps["ridge"].alpha_)
print("R2:", r2_score(y_ridge, pred_ridge))
print("MAE:", mean_absolute_error(y_ridge, pred_ridge))
print(coef.to_string(index=False))

print("\n" + "="*100)
print("10. FINAL VERDICT")
print("="*100)

mask_summary = mask_cv_log.groupby("model").agg(
    MAE=("MAE", "mean"),
    R2=("R2", "mean"),
).reset_index().set_index("model")

family_summary = family_cv_log.groupby("model").agg(
    MAE=("MAE", "mean"),
    R2=("R2", "mean"),
).reset_index().set_index("model")

def get_mae(summary, name):
    return float(summary.loc[name, "MAE"])

mask_geom = get_mae(mask_summary, "geometry_no_capacity")
mask_scalar = get_mae(mask_summary, "scalar_capacity")
mask_align = get_mae(mask_summary, "scalar_plus_alignment")

fam_geom = get_mae(family_summary, "geometry_no_capacity")
fam_scalar = get_mae(family_summary, "scalar_capacity")
fam_align = get_mae(family_summary, "scalar_plus_alignment")

print("Mask-heldout MAE geometry_no_capacity:", mask_geom)
print("Mask-heldout MAE scalar_capacity:", mask_scalar)
print("Mask-heldout MAE scalar_plus_alignment:", mask_align)
```

```
print("Mask scalar improvement:", mask_geom - mask_scalar)
print("Mask alignment improvement over scalar:", mask_scalar - mask_align)

print("\nFamily-heldout MAE geometry_no_capacity:", fam_geom)
print("Family-heldout MAE scalar_capacity:", fam_scalar)
print("Family-heldout MAE scalar_plus_alignment:", fam_align)
print("Family scalar improvement:", fam_geom - fam_scalar)
print("Family alignment improvement over scalar:", fam_scalar - fam_align)

ratio = df["ratio_VcR_over_VcBS"]
print("\nRatio Vc_R / Vc_BS mean:", ratio.mean())
print("Ratio Vc_R / Vc_BS std:", ratio.std())
print("Ratio coefficient of variation:", ratio.std() / abs(ratio.mean()))

if mask_scalar < mask_geom and fam_scalar < fam_geom:
    print("\nVERDICT: PASS — scalar projected capacity improves critical-threshold prediction over geometry-only.")
else:
    print("\nVERDICT: FAIL — scalar projected capacity does not beat geometry-only robustly.")

if mask_align < mask_scalar and fam_align < fam_scalar:
    print("ALIGNMENT: PASS — spectral-alignment features improve over scalar capacity.")
else:
    print("ALIGNMENT: NO — scalar capacity is already the main useful object or alignment is unstable.")
```

```
[progress] completed masks 15/180
[progress] completed masks 30/180
[progress] completed masks 45/180
[progress] completed masks 60/180
[progress] completed masks 75/180
[progress] completed masks 90/180
[progress] completed masks 105/180
[progress] completed masks 120/180
[progress] completed masks 135/180
[progress] completed masks 150/180
[progress] completed masks 165/180
[progress] completed masks 180/180

================================================================================================
1. RUN HEADER
================================================================================================
L: 64
N: 4096
K_LOW: 128
lambda1: 0.009630546655606143
lambda_max: 0.37549021458844867

================================================================================================
2. FIXED GEOMETRY SANITY
================================================================================================
  m  density  cluster_count  largest_cluster
128  0.03125             64                2

================================================================================================
3. FAMILY THRESHOLD SUMMARY
================================================================================================
                      n    R_mean     R_std   B0_mean    B0_std  Vc_BS_mean  Vc_BS_std  Vc_R_mean  Vc_R_std  ratio_mean  ratio_
family
blue_noise_dimers    30  0.106183  0.002768  3.432189  0.078997    0.291504   0.006530   0.090759  0.002429    0.311585   0.011
lowmode_biased_diag  30  0.138087  0.008458  5.210603  0.332743    0.192680   0.012433   0.069993  0.004230    0.363966   0.021
lowmode_biased_x     30  0.143193  0.009576  4.837252  0.297986    0.207454   0.012212   0.067549  0.004549    0.326197   0.021
random_dimers        30  0.130328  0.009720  4.401002  0.267303    0.228038   0.013945   0.074284  0.005420    0.325946   0.011
ring_dimers          30  0.167012  0.010683  6.068882  0.443105    0.165606   0.011802   0.057894  0.003736    0.350397   0.021
stripe_dimers        30  0.197039  0.005686  7.628295  0.150743    0.131140   0.002590   0.048916  0.001418    0.373041   0.009

================================================================================================
4. GLOBAL THRESHOLD DIAGNOSTICS
================================================================================================
corr(Vc_R, Vc_BS): 0.9581230919858064
corr(log Vc_R, log Vc_BS): 0.9599848701034337
MAE Vc_R vs Vc_BS: 0.13450481151290433
MAE log Vc_R vs log Vc_BS: 1.076866315412275

ratio Vc_R / Vc_BS:
count    180.000000
mean       0.341856
std        0.028654
min        0.280871
25%        0.317341
50%        0.341643
75%        0.367471
max        0.406103
```